STA2201: Methods of Applied Statistics II

Philip Loewen

2025W2

Contents

1 Review: Matrices and Decompositions **1**

2 Review: MVN distribution **7**
2.1 Transformations . . . 7
2.2 Independence . . . 8
2.3 Geometry of MVN . . . 8
2.4 Conditional Distribution . . . 9
2.5 Block Matrices . . . 10
2.6 Partial Correlation . . . 11
2.7 Maximum Likelihood Estimates . . . 12

3 Curse of Dimensionality **13**
3.1 High-dimensional geometry . . . 13

4 Principal Component Analysis (PCA) **17**
4.1 Dimension Reduction . . . 17
4.2 PCA . . . 17
4.3 Practical Aspects of PCA . . . 21
4.4 Power Iteration . . . 22
4.5 PCA: Minimum Reconstruction Error View . . . 23
4.6 PCA via SVD . . . 25
4.7 PCA: Low-Rank Matrix Approximation View . . . 26
4.8 Low-Rank Matrix Approximation (LRMA) . . . 27
4.9 Low-Rank Matrix Completion . . . 28
4.10 Soft Low-Rank Approximation . . . 29
4.11 Proximal Operators . . . 30
4.12 Proximal Gradient Descent . . . 32
4.13 Sparse PCA . . . 32
4.14 Application: Shape Analysis (from ESLII) . . . 34

5 Non-Linear Dimension Reduction **35**
5.1 Kernel PCA 35
5.2 Local Dimension Reduction Methods 36
5.3 Perturbation Theory for PCA 37

6 Clustering **40**
6.1 K-Means Clustering 40
6.2 Applications of K-Means 41
6.3 Limitations and Modifications of K-means 41
6.4 Soft Clustering 42
6.5 Cluster Scatter 42
6.6 Choosing K 43
6.7 Hierarchical Clustering 44
6.8 Gaussian Mixture Model (GMM) 45
6.9 EM Algorithm for GMM 45
6.10 EM Algorithm: Motivation 47
6.11 EM for GMM (Summary) 48
6.12 Spectral Clustering 49
6.13 Graph Laplacian 50
6.14 Spectral Clustering Algorithm 50
6.15 Why Spectral Clustering Works 51
6.16 Graph-Cut Point of View (Assume $K = 2$) 51
6.17 Comments 53

Chapter 1

Review: Matrices and Decompositions

Definition (Vector product). We can compute the product of vectors in two ways:

1. The inner product: If $a, b \in \mathbb{R}^n$, $\langle a, b\rangle = a^T b = \sum a_i b_i$.

2. The outer product: If $a \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, $ab^T = \begin{pmatrix} a_1 b_1 & \cdots & a_1 b_m \\ \vdots & \ddots & \vdots \\ a_n b_1 & \cdots & a_n b_m \end{pmatrix}$

Definition (Matrix product). Let $A \in \mathbb{R}^{n\times p}$ and $B \in \mathbb{R}^{p\times m}$. We have two views to the matrix product:

1. Inner product:
$$A = \begin{pmatrix} — & a_1^T & — \\ & \vdots & \\ — & a_n^T & — \end{pmatrix}, B = \begin{pmatrix} | & & | \\ b_1 & \cdots & b_n \\ | & & | \end{pmatrix}$$
$$(AB)_{ij} = a_i^T b_j.$$

2. Outer product:
$$A = \begin{pmatrix} | & & | \\ a_1 & \cdots & a_n \\ | & & | \end{pmatrix} B = \begin{pmatrix} — & b_1^T & — \\ & \vdots & \\ — & b_n^T & — \end{pmatrix},$$
$$AB = \sum_{j=1}^{p} a_j b_j^T$$

. Note that the $a_j b_j^T$ are $n \times m$ matrices with rank 1.

Theorem (Rank factorization). Let $A \in \mathbb{R}^{n\times p}$ with $\text{rank}(A) = r$. Then there exist $C \in \mathbb{R}^{n\times r}$, $F \in \mathbb{R}^{p\times r}$ such that C, F are full column rank and $A = CF^T = \sum_{i=1}^{r} c_i f_i^T$.

Theorem (QR decomposition). Any matrix $A \in \mathbb{R}^{n\times p}$ can be decomposed as $A = QR$ where $Q \in \mathbb{R}^{n\times p}$ is orthogonal and $R \in \mathbb{R}^{p\times p}$ is upper triangular. R can be obtained through the Gram-Schmidt process:

- $e_1 = a_1, \quad q_1 = \dfrac{e_1}{||e_1||}.$
- $e_2 = a_2 - \langle a_2, q_1\rangle \cdot q_1, \quad q_2 = \dfrac{e_2}{||e_2||}.$

  $\vdots$
- $e_k = a_k - \displaystyle\sum_{i=1}^{k-1} \langle a_k, q_i\rangle \cdot q_j, \quad q_k = \dfrac{e_k}{||e_k||}.$

This decomposition gives an orthogonal basis for the span of the columns of A. Furthermore, the rank of A is reflected in R. If $rank(A) = r$ then R will have r non-zero rows. If $r < p$ then we can truncate Q and R.

Example (Linear regression). To minimize $X\beta = y$, we set

$$\begin{aligned}\hat{\beta} &= (X^TX)^{-1}X^Ty \\ &= (R^TQ^TQR)^{-1}R^TQ^Ty \\ &= (R^TR)^{-1}R^TQ^Ty \\ &= R^{-1}Q^Ty\end{aligned}$$

Hence $R\hat{\beta} = Q^Ty$, which can be easily solved.

Theorem (Eigen decomposition). Let $A \in \mathbb{R}^{n\times n}$ with eigenvalues λ_i and corresponding eigenvectors u_i. Then $A = U\Lambda U^T$ where

$$U = \begin{pmatrix} | & & | \\ u_1 & \cdots & u_n \\ | & & | \end{pmatrix}, \qquad \Lambda = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 & 0 \\ 0 & \lambda_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \lambda_{n-1} & 0 \\ 0 & 0 & \cdots & 0 & \lambda_n \end{pmatrix}$$

Using the outer product view, we have $A = \displaystyle\sum_{i=1}^{n} \lambda_i u_i u_i^T$.

Properties. Some properties of ED:

- $A = \displaystyle\sum_{i=1}^{n} \lambda_i u_i u_i^T \implies Au_j = u_j \sum_{i=1}^{n} \lambda_i u_i u_i^T = \lambda_j u_j.$

- ED is not unique.

  If $\lambda_1 = \cdots = \lambda_k$ then we can take $(\tilde{u_1} \cdots \tilde{u_r}) = (u_1 \cdots u_r)Q$, where $Q \in \mathbb{R}^{r \times r}$ is some orthogonal matrix.

  If u_i is an eigenvector then cu_i is also an eigenvector for any scalar $c \neq 0$.

- The rank of A is the number of non-zero eigenvalues (with multiplicity)

Definition (Definite matrices). We say that a symmetric matrix $A \in \mathbb{R}^{n \times n}$ is

- positive definite ($A \succ 0$) if $v^T A v > 0 \forall v \in \mathbb{R}^n$.
- positive semi-definite ($A \succeq 0$) if $v^T A v \geq 0 \forall v \in \mathbb{R}^n$.

Theorem. Let $A \in \mathbb{R}^{n \times n}$ and let $\{\lambda_i\}_{i=1}^n$ be its eigenvalues with duplicity. Then

$$A \succ 0 \iff \lambda_i > 0, \quad (i = 1, \cdots, n).$$

Similarly

$$A \succeq 0 \iff \lambda_i \geq 0, \quad (i = 1, \cdots, n).$$

Proof. ($\implies$) Let $A \succ 0$. Then $v^T A v > 0 \forall v \in \mathbb{R}^n$. Take $v = u_i$ where u_i is the eigenvector corresponding to λ_i. Then we have $\lambda_i u_i u_i^T > 0$. Hence $\lambda_i > 0$.
($\impliedby$) If $\lambda_i > 0$ for all $i = 1, \cdots n$, then $v^T A v = v^T U \Lambda U^T v$. If we set $y = U^T v$, then $v^T A v = y^T \Lambda y = \sum_{i=1}^n \lambda_i y_i^2 > 0$.
A similar proof holds for the second equivalence. □

This decomposition allows us to easily compute powers of A:

- $A^k = U \Lambda^k U^T$.
- If $A \succ 0$, $A^{-1} = U \Lambda^{-1} U^T$.
- If $A \succeq 0$, $A^{\frac{1}{2}} = U \Lambda^{\frac{1}{2}} U^T$.

Remark. This implies that powers of A have the same eigenvectors as A, with new eigenvalues taken as the power of the corresponding original eigenvalues.

Definition (Projections). $P \in \mathbb{R}^{n \times n}$ is an orthogonal projection matrix if $P^T = P = P^2$.

Properties. Projection matrices have some nice properties:

- $P = P^2 \implies U \Lambda U^T = U \Lambda^2 U^T \implies \lambda_i = \lambda_i^2 \implies \lambda_i \in \{0, 1\}$.
- $P \succeq 0$.
- Diagonal elements of P are bounded by 0 and 1.

Proof.

$$P = P^2 = P^T P \implies p_{ii} = \sum_{k=1}^{n} p_{ki}^2 \geq p_{ii}^2 \geq 0$$

Since $p_{ii} \geq p_{ii}^2$, $p_{ii} \leq 1.22$ □

Example. Let $C(A) = \text{span}(a_1, \cdots, a_p) = \{A_v : v \in \mathbb{R}^p\}$ be the column space of A.

- $U \in \mathbb{R}^{n\times p} \implies U(U^TU)^{-1}U^T = P$ projects onto $C(U)$.

  This can easily be seen by computing

  $$\hat{v} = \underset{v\in\mathbb{R}^p}{\text{argmin}}\, ||y - Uv||^2 = (U^TU)^{-1}U^Ty$$

  which implies $\hat{y} = U\hat{v} = U(U^TU)^{-1}U^Ty = Py$. Hence $P = U(U^TU)^{-1}U^T$.

- If U is column orthogonal, $P = UU^T$ and $P_\perp = I - UU^T$ which projects onto $C(U_\perp)$ where $U_\perp \in \mathbb{R}^{n\times(n-p)}$ is the orthogonal complement to $U \in \mathbb{R}^{n\times p}$ if the extended $\tilde{U} = (U, U_\perp) \in \mathbb{R}^{n\times n}$ is orthogonal.

Definition (Centering operator). Let

$$X \in \mathbb{R}^{n\times p} = \begin{pmatrix} — & x_1^T & — \\ & \vdots & \\ — & x_n^T & — \end{pmatrix} = \begin{pmatrix} | & & | \\ f_1 & \cdots & f_p \\ | & & | \end{pmatrix}.$$

Then the centering operator is the matrix $C \in \mathbb{R}_{n\times n}$ such that

$$CX = \tilde{X} = \begin{pmatrix} — & (x_1 - \overline{x})^T & — \\ & \vdots & \\ — & (x_n - \overline{x})^T & — \end{pmatrix} = \begin{pmatrix} | & & | \\ (f_1 - \overline{f_1}) & \cdots & (f_p - \overline{f_p}) \\ | & & | \end{pmatrix}$$

where $\overline{x} = \dfrac{1}{n}\displaystyle\sum_{i=1}^{n} x_i = \dfrac{1}{n}X^T\mathbf{1}$

Proposition. $C = I - \dfrac{1}{n}\mathbf{1}\mathbf{1}^T$ is the centering operator.

Proof. $\begin{pmatrix} (x_1 - \overline{x})^T \\ \vdots \\ (x_n - \overline{x})^T \end{pmatrix} = X - \mathbf{1}\overline{x}^T = X - \dfrac{\mathbf{1}\mathbf{1}^T}{n}X = \left(I - \dfrac{\mathbf{1}\mathbf{1}^T}{n}\right)X = CX.$

$$C = \begin{pmatrix} 1 - \frac{1}{n} & \frac{-1}{n} & \cdots & \frac{-1}{n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{-1}{n} & \frac{-1}{n} & \cdots & 1 - \frac{1}{n} \end{pmatrix}$$ □

Properties. C is an orthogonal projection operator. If $U = \dfrac{\mathbf{1}}{\sqrt{n}}$ then $C = I - UU^T$ projects onto $C(U_\perp)$.

Definition (Covariance matrix). The covariance matrix S of X is

$$S = \frac{\tilde{X}^T\tilde{X}}{n-1} = \frac{X^TCX}{n-1} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})(x_i - \overline{x})^T \in \mathbb{R}^{p\times p}.$$

- $v^TSv = v^T\dfrac{\tilde{X}^T\tilde{X}}{n-1}v = \dfrac{yy^T}{n-1} = \dfrac{||y||}{n-1} \geq 0$. Hence S is PSD.
- $v^TSv = \dfrac{||CXv||^2}{n-1} = \dfrac{||Cz||^2}{n-1} = \mathrm{Var}(z) = \mathrm{Var}(Xv)$.
- $\hat{\beta} = (X^TX)^{-1}X^Ty = \left(\dfrac{X^TX}{n-1}\right)\dfrac{X^Ty}{n-1} = S_X^{-1}S_{Xy}$.

Theorem. Let $A \in \mathbb{R}^{n\times p}$. Then there exist $U \in \mathbb{R}^{n\times n}$, $V \in \mathbb{R}^{p\times p}$, $D \in \mathbb{R}^{n\times p}$ where U, V are orthogonal and D is diagonal with $d_1 \geq d_2 \geq \cdots \geq d_p \geq 0$ such that

$$A = UDV^T = \overbrace{\begin{pmatrix} | & & | \\ u_1 & \cdots & u_n \\ | & & | \end{pmatrix}}^{n\times n} \overbrace{\begin{pmatrix} d_1 & & \\ & \ddots & \\ & & d_p \\ & & \\ & & \end{pmatrix}}^{n\times p} \overbrace{\begin{pmatrix} — & v_1^T & — \\ & \vdots & \\ — & v_n^T & — \end{pmatrix}}^{p\times p} \quad \left| \begin{array}{c} UU^T = U^TU = I_n \\ V^TV = VV^T = I_p \\ d_1 \geq d_2 \geq \cdots \geq d_p \geq 0 \end{array} \right.$$

$$= \underbrace{\begin{pmatrix} | & & | \\ u_1 & \cdots & u_p \\ | & & | \end{pmatrix}}_{n\times p} \underbrace{\begin{pmatrix} d_1 & & \\ & \ddots & \\ & & d_p \end{pmatrix}}_{p\times p} \underbrace{\begin{pmatrix} — & v_1^T & — \\ & \vdots & \\ — & v_n^T & — \end{pmatrix}}_{p\times p} \quad \left| \begin{array}{c} UU^T = I_n \\ V^TV = VV^T = I_p \\ d_1 \geq d_2 \geq \cdots \geq d_p \geq 0 \end{array} \right.$$

The second form is known as the compact form.

Remark. The SVD and ED have the same properties. In fact, the SVD is a generalization of ED where $A \in \mathbb{R}^{n\times n}$ is PSD, and if $\lambda_i < 0$, we can take $d_i = -\lambda_i, u_i = u_i$ and $v_i = -u_i$.

Remark. If $A \in \mathbb{R}^{n\times p}$ then the v_i are the eigenvectors of A^TA and d_i^2 are the corresponding eigenvalues. If $n \geq p$ then u_i are the eigenvectors of AA^T and d_i^2 are the corresponding eigenvalues (with some extra 0 eigenvalues).

Definition (Norms). There are several types of norms. For a vector $v \in \mathbb{R}^n$:

- $\ell_1 : ||v||_1 = \sum_{i=1}^{n} |v_i|$.

- $\ell_2 : ||v||_2 = \sum_{i=1}^{n} v_i^2$.

And for a matrix $A \in \mathbb{R}^{n \times p}$:

- Frobenius norm: $||A||_F^2 = \sum_{i=1}^{n} \sum_{j=1}^{p} A_{ij}^2 = \text{tr}(A^T A) = \sum_{i=1}^{\min(n,p)} d_i(A)^2$.
- Nuclear norm: $||A||_* = \sum_{i=1}^{\min(n,p)} d_i(A)$.

Definition (Gradients). If $f : \mathbb{R}^n \to \mathbb{R}$, then

$$\nabla_x f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{pmatrix}$$

If $f : \mathbb{R}^{n \times p} \to \mathbb{R}$, then

$$\nabla_A f(A) = \begin{pmatrix} \frac{\partial f(A)}{\partial A_{11}} & \cdots & \frac{\partial f(A)}{\partial A_{m1}} \\ \vdots & & \\ \frac{\partial f(A)}{\partial A_{n1}} & \cdots & \frac{\partial f(A)}{\partial A_{nm}} \end{pmatrix}$$

Example.

- $\nabla_x \langle a, x \rangle = a$.
- $\nabla_x ||x||^2 = 2x$.
- If S is symmetric, $\nabla_x x^T S x = 2Sx$.
- $\nabla_A ||A||_F^2 = 2A$.
- $\nabla_A \text{tr}(AB^T) = B$.

Chapter 2

Review: MVN distribution

Definition (Multivariate Normal). A random vector $x = (x_1, x_2, \ldots, x_p)^T$ is a p-dimensional multivariate normal (MVN) if and only if $v^T X$ is univariate normal for all $v \in \mathbb{R}^p$. We write $X \sim N_p(\mu, \Sigma)$ where $\mathrm{E}(X) = \mu \in \mathbb{R}^p$ is the mean and $\mathrm{Var}(X) = \Sigma \in \mathbb{R}^P p \times p$ is the variance. The density function of X is

$$f_X(x) = \frac{1}{(2\pi)^{p/2} \det(\Sigma)^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

and the characteristic function is

$$\phi_X(t) = e^{it^T\mu - \frac{1}{2}t^T \Sigma t}.$$

2.1 Transformations

Properties.

1. If $X \sim N_p(\mu, \Sigma), A \in \mathbb{R}^{q\times p}, B \in \mathbb{R}^q$ and $y = AX + B$ then $y \sim N_q(A\mu + B, A\Sigma A^T)$.

   Proof. $v^T y = v^T(AX + B) = v^T AX + v^T B = \tilde{v}^T X + \tilde{B}$. By definition, $\tilde{v}^T X$ is normal, hence y is q-dimensional MVN. □

2. If $X \sim N_p(0, I)$ then $y = \Sigma^{1/2} X + \mu \sim N_p(\mu, \Sigma)$.

3. If $X \sim N_p(\mu, \Sigma)$ and $y = \Sigma^{-1/2}(X - \mu)$ then $y \sim N_p(0, I)$.

4. If $X \sim N_p(\mu, \Sigma)$ then $X_i \sim N(\mu_i, \Sigma_{ii})$.

   Proof. This follows from property 1 with $A = (0, \ldots, 0, 1, 0, \ldots, 0)^T \in \mathbb{R}^{p\times 1}$ where the i-th entry is 1 and all others are 0. □

5. If $X \sim N_p(\mu, \Sigma)$, $\mathcal{I} = \{i_1, \ldots, i_i\} \subseteq \{1, \ldots, p\}$ and $X_{\mathcal{I}} = (X_{i_1}, X_{i_2}, \ldots, X_{i_r})$ then $X_{\mathcal{I}} \sim N_r(\mu_{\mathcal{I}}, \Sigma_{\mathcal{I}\mathcal{I}}))$.

Proof. This follows from property 1 with $A = \begin{pmatrix} 0 & \dots & \overbrace{1}^{i_1\text{entry}} & \dots & 0 & \dots \\ 0 & \dots & 0 & \dots & \underbrace{1}_{i_2\text{entry}} & \dots \end{pmatrix}$ i.e. in the k-th row, the i_k-th entry is 1 and all others are 0. □

2.2 Independence

Theorem. Let $(X, Y)^T$ be $p+q$-dimensional MVN where X and Y are p and q dimensional respectively. X and Y are independent if and only if they are uncorrelated.
In other words,

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim N_{p+q}\left(\begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}, \begin{pmatrix} \Sigma_X & 0 \\ 0 & \Sigma_Y \end{pmatrix}\right)$$

implies that $X \sim N_p(\mu_X, \Sigma_X)$ and $Y \sim N_q(\mu_Y, \Sigma_Y)$ are independent

2.3 Geometry of MVN

Proposition. Contours of f_X are $\{x \in \mathbb{R}^p : d_\Sigma(x, \mu) = c)$ where $d_\Sigma(x, \mu) = (x-\mu)^T\Sigma^{-1}(X-\mu)$ is Mahalanobis distance.

Proof. If we set $f_X(x) = c'$, then

$$f_X(x) = \frac{1}{(2\pi)^{p/2}\det(\Sigma)^{1/2}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)} = c'.$$

$$\implies d_\Sigma(x, \mu) = (x-\mu)^T\Sigma^{-1}(x-\mu) = c.$$

□

Proposition. Contours of f_X are ellipsoids.

Proof. Using the SVD, $\Sigma = U\Lambda U^T$ and set $y = U^T(X-\mu) \sim N_p(0, \Lambda)$. Then

$$d_\Lambda(y, 0) = c$$
$$y^T\Lambda y = \sum_{i=1}^{p} \frac{y_i^2}{\lambda_i} = c$$

Hence $\{y \in \mathbb{R}^p : d_\Lambda(y, 0) = c\}$ is an ellipsoid with semi-axes $\sqrt{\lambda_1 c}, \dots, \sqrt{\lambda_p c}$.
Therefore $X = Uy + \mu$ is just a rotation and shift of the ellipsoid, giving semi-axes aligned with the columns of U (i.e. eigenvectors of Σ). □

Proposition. For a random vector $X \sim N_p(\mu, \Sigma)$, $(X-\mu)^T\Sigma^{-1}(X-\mu) \sim \chi^2(p)$.

Proof.

$$\begin{aligned}(X-\mu)^T\Sigma^{-1}(X-\mu) &= (X-\mu)^T\Sigma^{-1/2}\Sigma^{-1/2}(X-\mu)\\ &= Y^TY\end{aligned}$$

where $Y \sim N_p(0, I)$. Since $Y^TY = \sim_{i=1}^{p} y_i^2$ where $y_i \sim N_1(0,1)$ we get $Y^TY \sim \chi^2(p)$. □

Corollary. This allows us to compute $P((X-\mu)^T\Sigma^{-1}(X-\mu) < t)$ and draw confidence ellipses.

2.4 Conditional Distribution

Proposition. Let

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim N_{p+q}\left(\begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}, \begin{pmatrix} \Sigma_X & \Sigma_{XY} \\ \Sigma YX & \Sigma_Y \end{pmatrix}\right)$$

and denote $Z = y - \Sigma_{YX}\Sigma_X^{-1}X$. Then

1. $X \sim N_p(\mu_X, \Sigma_X)$.
2. $Z \sim N_q(\mu_Y - \Sigma_{YX}\Sigma_X^{-1}X, \Sigma_Y - \Sigma_{YX}\Sigma_X^{-1}\Sigma_{XY})$.

Proof. Let $B = \begin{pmatrix} -\Sigma_{YX}\Sigma_X^{-1} & I \end{pmatrix}$. Then

$$\begin{aligned}B(X,Y)^T &= Z\\ B\mu &= \begin{pmatrix} -\Sigma_{YX}\Sigma_X^{-1} & I \end{pmatrix}\begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}\\ &= \mu_Y - \Sigma_{YX}\Sigma_X^{-1}X\\ B\Sigma B^T &= \begin{pmatrix} -\Sigma_{YX}\Sigma_X^{-1} & I \end{pmatrix}\begin{pmatrix} \Sigma_X & \Sigma_{XY} \\ \Sigma YX & \Sigma_Y \end{pmatrix}\begin{pmatrix} -\Sigma_Y X\Sigma_X^{-1} \\ I \end{pmatrix}\\ &= \begin{pmatrix} -\Sigma_{YX}\Sigma_X^{-1}\Sigma_X + \Sigma_{YX} & -\Sigma_{YX}\Sigma_X^{-1}\Sigma_{XY} + \Sigma_Y \end{pmatrix}\begin{pmatrix} -\Sigma_Y X\Sigma_X^{-1} \\ I \end{pmatrix}\\ &= \begin{pmatrix} 0 & -\Sigma_{YX}\Sigma_X^{-1}\Sigma_{XY} + Y \end{pmatrix}\begin{pmatrix} -\Sigma_Y X\Sigma_X^{-1} \\ I \end{pmatrix}\\ &= -\Sigma_{YX}\Sigma_X^{-1}\Sigma_{XY} + \Sigma_Y.\end{aligned}$$

□

Proposition. X and Z are independent.

Proof. Let $A = \begin{pmatrix} I & 0 \end{pmatrix}$. Then $X = A \begin{pmatrix} X \\ Y \end{pmatrix}$ and

$$\begin{aligned}
\operatorname{Cov}(X, Z) &= A \operatorname{Var} \begin{pmatrix} X \\ Y \end{pmatrix} B^T \\
&= \begin{pmatrix} I & 0 \end{pmatrix} \begin{pmatrix} \Sigma_X & \Sigma_{XY} \\ \Sigma YX & \Sigma_Y \end{pmatrix} \begin{pmatrix} -\Sigma_Y X \Sigma_X^{-1} \\ I \end{pmatrix} \\
&= \begin{pmatrix} \Sigma_X & \Sigma_{XY} \end{pmatrix} \begin{pmatrix} -\Sigma_{YX} \Sigma_X^{-1} \\ I \end{pmatrix} \\
&= \Sigma_{XY} + \Sigma_{XY} = 0.
\end{aligned}$$

□

Theorem. $y|X = x \sim N_q(\mu_Y + \Sigma_{YX}\Sigma_X^{-1}(x - \mu), \Sigma_Y - \Sigma_{YX}\Sigma_X^{-1}\Sigma_{XY})$.

Proof. We have $Y = Z + \Sigma_{YX}\Sigma_X^{-1}X$. Then clearly $y|X = x$ is MVN and

$$\begin{aligned}
\mathrm{E}[Y|X = x] &= \mathrm{E}[z] + \Sigma_{YX}\Sigma_X^{-1}x \\
&= \mu_Y - \Sigma_{YX}\Sigma_X^{-1}\mu_X + \Sigma_{YX}\Sigma_X^{-1}x \\
&= \mu_Y + \Sigma_{YX}\Sigma_X^{-1}(x - \mu_X)
\end{aligned}$$

$$\operatorname{Var}[Y|X = x] = \operatorname{Var}[Z] = \Sigma_Y - \Sigma_{YX}\Sigma_X^{-1}\Sigma_{XY}.$$

□

Remark (Connection to regression). Let $y = (y_1, \ldots, y_n)^T \in \mathbb{R}^n$, $X = (f_1, \ldots, f_p) = (x_1, \ldots, x_n)^T$. Then $e = y - \hat{y} = y - X\hat{\beta} = y - XS_X^{-1}S_{XY}$.
We remark that e and Z are very similar, with e using the observed variance.

2.5 Block Matrices

Let $M \in \mathbb{R}^{(p+q)\times(p+q)}$ with

$$M = \begin{pmatrix} A & B \\ B & C \end{pmatrix}, A \text{ and } C \text{ invertible }.$$

Definition (Schur-Complement). The Schur-Complement of C is $M/C = A - BC^{-1}B^T \in \mathbb{R}^{p\times p}$. Similarly, $M/A = C - BA^{-1}B^T \in \mathbb{R}^{p\times p}$.

Lemma (Block Inverse). Using the Schur-complement,

$$M^{-1} = \begin{pmatrix} (M/C)^{-1} & -A^{-1}B(M/A))^{-1} \\ C^{-1}B(M/C))^{-1} & (M/A)^{-1} \end{pmatrix}.$$

Properties. $M \succ 0$ if and only if $A \succ 0$ and $M/A \succ 0$.
$M \succ 0$ if and only if $C \succ 0$ and $M/C \succ 0$.

2.6 Partial Correlation

Consider $X \sim N_p(\mu, \Sigma)$ and partition it as

$$X = \begin{pmatrix} X_A \\ X_B \end{pmatrix} \sim N_{p_A+p_B}\left(\begin{pmatrix} \mu_A \\ \mu_B \end{pmatrix}, \begin{pmatrix} \Sigma_A & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_B \end{pmatrix}\right)$$

where $p_A + p_B = p$.
Then $X_A|X_B = x \sim N_{p_A}(\ldots, \Sigma_{A\cdot B} = \Sigma_A - \Sigma_{AB}\Sigma_B^{-1}\Sigma_{BA})$ where $\Sigma_{A\cdot B}$ is called partial correlation.
Let

$$X_A = \begin{pmatrix} X_1 \\ \vdots \\ X_{p_A} \end{pmatrix}$$

and set $z_i = X_1 - \Sigma_{X_iX_B}\Sigma_B^{-1}X_B$. Then, $(\Sigma_{A\cdot B})_{ij} = \mathrm{Cov}(z_i, z_j)$ for $i, j = 1, \ldots, p_A$.

Proof. We know $\mathrm{Cov}(z_i, z_j) = \Sigma_{X_iX_j} - \Sigma_{X_iX_B}\Sigma_{X_B}^{-1}\Sigma_{X_BX_j}$ where $\Sigma_{X_iX_j} = (\Sigma_A)_{ij}, \Sigma_{X_iX_B} = (\Sigma_{AB})_{i\cdot}$ and $\Sigma_{X_BX_j} = (\Sigma_{BA})_{\cdot j}$. Thus,

$$\mathrm{Cov}(z_i, z_j) = (\Sigma_A - \Sigma_{AB}\Sigma_B^{-1}\Sigma_{BA})_{ij}.$$

□

Definition (Precision Matrix). The precision matrix is $P = \Sigma^{-1}$.

Remark. Since $\Sigma_{A\cdot B} = \Sigma/\Sigma_B$,

$$P = \begin{pmatrix} (\Sigma_{A\cdot B})^{-1} & * \\ * & * \end{pmatrix}.$$

Definition (Partial Correlation). The partial correlations $\rho_{ij\cdot B} = \mathrm{Cor}(z_i, z_j)$ measures the relationship between x_i and x_j after removing the effect of X_B.

Example. If $p_A = 2$, we have

$$P = \Sigma^{-1} = \begin{pmatrix} (\Sigma_{A\cdot B})^{-1} & * \\ * & * \end{pmatrix}, \qquad (\Sigma_{AB})^{-1} = \frac{1}{a_11a_22 - a_{12}a_{21}}\begin{pmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{pmatrix}$$

Therefore,

$$\rho_{12\cdot B} = \frac{\mathrm{Cov}(z_1, z_2)}{\sqrt{\mathrm{Var}(z_1)\,\mathrm{Var}(z_2)}} = \frac{-p_{12}}{\sqrt{p_{11}\cdot p_{22}}}.$$

Hence if $p_{12} = p_{21} = 0$ then $X_1 \perp X_2|X_B$.
More generally, if

$$X_A = \begin{pmatrix} X_i & X_j \end{pmatrix}^T, \quad X_B = \begin{pmatrix} X_1 & \cdots & X_{i-1} & X_{i+1} & \cdots & X_{j-1} & X_{j+1} & \cdots \end{pmatrix}^T$$

then,

$$\rho_{ij\cdot B} = -\frac{p_{ij}}{\sqrt{p_{ii}p_{jj}}}.$$

2.7 Maximum Likelihood Estimates

Let $x_1, \ldots, x_n \in \mathbb{R}^p$ with

$$\ell(x_1, \ldots, x_n; \mu, \Sigma) = -\frac{n \cdot p}{2} \log(2\pi) - \frac{n}{2} \log \det(\Sigma) - \frac{1}{2} \sum_{i=1}^{n} (x_i - \mu)^T \Sigma (x_i - \mu).$$

Then,

$$\hat{\mu} = \overline{x} = \frac{1}{n} \Sigma_{i=1}^{n} x_i \tag{1}$$

$$\ell(x_1, \ldots, x_n; \overline{x}, \Sigma) = -\frac{n}{2} \log \det(\Sigma) - \frac{n}{2} \mathrm{tr}(\Sigma^{-1} S) + \ldots \tag{2}$$

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^{n} (x_i - \overline{x})^T (x_i - \overline{x})^T \tag{3}$$

where $S = \dfrac{1}{n} \displaystyle\sum_{i=1}^{n} (x_i - \overline{x})^T (x_i - \overline{x})^T$.

Proof. To prove (2) it suffices that

$$\begin{aligned}
\sum_{i=1}^{n} \mathrm{tr}((x_i - \overline{x})^T \Sigma^{-1} (x_i - \overline{x})) &= \sum_{i=1}^{n} \mathrm{tr}(\Sigma^{-1} (x_i - \overline{x})(x_i - \overline{x})^T) \\
&= \mathrm{tr}(\Sigma^{-1} \sum_{i=1}^{n} (x_i - \overline{x})(x_i - \overline{x})^T) \\
&= \mathrm{tr}(n \Sigma^{-1} S).
\end{aligned}$$

(3) follows by computing the gradient:

$$\nabla_p \left(\frac{n}{2} \log \det(P) - \frac{n}{2} \mathrm{tr}(PS) \right) = \frac{n}{2} P^{-1} - \frac{n}{2} S.$$

Therefore, $\hat{P} = S^{-1}$ and $\hat{\Sigma} = S$. □

One can show that $\hat{\mu} \sim N_p(\mu, \Sigma/n)$, $n\hat{\Sigma} = (n-1)S \sim W_p(\Sigma, n-1)$ and that $\hat{\mu}$ and $\hat{\Sigma}$ are independent.

Definition (Wishart Distribution). If $x_1, \ldots, x_p \sim N_p(0, \Sigma)$ and

$$X = \begin{pmatrix} — x_1^T — \\ \vdots \\ — x_n^T — \end{pmatrix}$$

then $M = X^T X$ follows a Wishart distribution. We write $M \sim W_p(\Sigma, n)$.

Remark. $W_p(\Sigma, n)$ is a distribution on PSD matrices of size $p \times p$.

Proposition. The Wishart distribution generalizes χ^2.

Proof. If $p = 1$, $X \in \mathbb{R}^p$ with $x_i \sim N(0, \sigma^2$, then $X^T X = \displaystyle\sum_{i=1}^{n} x_i^2 \sim \sigma^2 \chi^2(n) = W_1(\sigma^2, n)$. □

Chapter 3

Curse of Dimensionality

We often face issues in high-dimensions such as interpretability and visualization, noise level, computations issues and combinatoric issues.

Example. Consider a medical study with $n = 1000$ patients with p symptoms, each with 3 levels. Then for each $x_i \in \{0, 1, 2\}^p$, there are 3^p options for syptom combinations.
On average there will be $n/(3^p)$ patients per combination.

- $p = 3$ gives $\dfrac{1000}{3^3} \approx 37$.
- $p = 6$ gives $\dfrac{1000}{3^6} \approx 1.4$.

Therefore we will need a very large sample size to draw meaningful conclusion.

Many methods in statistics rely on pairwise distances between observations. However, distances are not reliable in high dimensions.

3.1 High-dimensional geometry

3.1.1 Nobody can hear you scream in high-dimensions

Assume $n = 100$ points in p-dimensional hypercube $H_p(1) = [0, 1]^p$. Then the volume per observation is $1/n = 0.01$.

Therefore the hypercube occupying this space has side $\sqrt[p]{\dfrac{1}{n}}$. For high dimensions and not huge observation counts this grows to 1 really quickly.

3.1.2 Orange Peel

Suppose $X_1, \ldots X_n \sim \text{Unif}(B_p(1))$. The median distance from the origin to the closest point is $\left(1 - \left(\dfrac{1}{2}\right)^{1/n}\right)^{1/p}$.

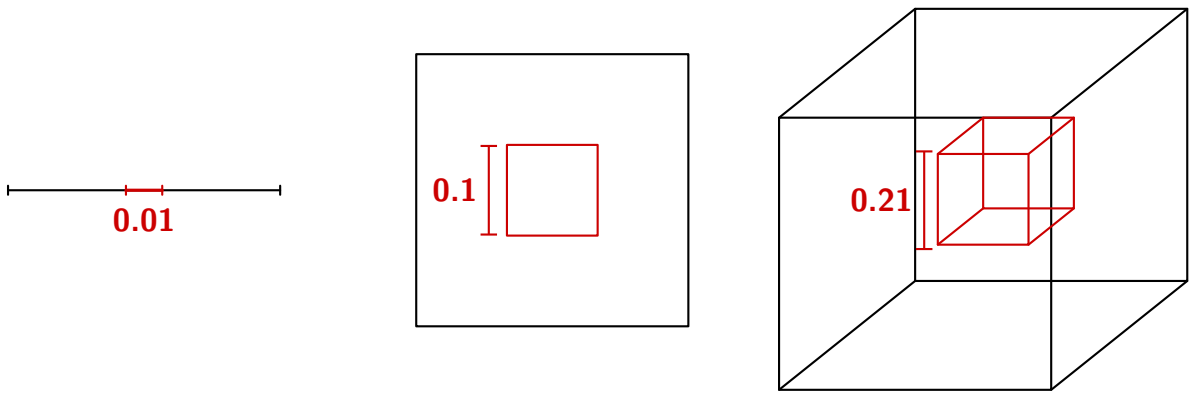


Figure 3.1: Examples of hypercubes occupying an area of 0.01 and the length of their vertices within $H_p(1)$, $p = 1, 2, 3$.

Proof. Recall the volume of $B_p(r)$ is $\dfrac{\pi^{p/2}}{\Gamma(p/2+1)} r^p$. Therefore, if r is the median of the distribution of minimum of the X_i,

$$
\begin{aligned}
&P(\min_{i=1,\dots,n} ||X_i|| \geq r) = \frac{1}{2} \\
&P(||X_i|| \leq r) = r^p = \frac{\mathrm{Vol}(B_p(r)}{\mathrm{Vol}(B_p(1)} \\
&P(||X_i|| \geq r) = 1 - r^p \\
&P(\min_{i=1,\dots,n} ||X_i|| \geq r) = \prod_{i=1}^{n} (1 - r^p) = \frac{1}{2} \\
&1 - r^p = \left(\frac{1}{2}\right)^{\frac{1}{n}} \\
&r = \left(1 - \left(\frac{1}{2}\right)^{1/n}\right)^{1/p}
\end{aligned}
$$

□

Remark. We see that $\lim_{p\to\infty} r = 1$, i.e. as $p \to \infty$, the distance from the closest point to the origin goes to 1. Therefore the "nearest neighbour" loses meaning in high-dimensional spaces.

3.1.3 Spiked hypercube

Suppose $X \sim \mathrm{Unif}(H_p(2))$ where $H_p(2) = [-1, 1]^p$. The probability mass is concentrated in the corners of the hypercube, i.e. $P(X \in B_p(1)) \to 0$ as $p \to \infty$.

Proof.

$$P(X \in B_p(1)) = \frac{\mathrm{Vol}(B_p(1)}{\mathrm{Vol}(H_p(2)} = \frac{\pi^{p/2}\Gamma\left(p/2+1\right)^{-1}}{2^p}$$
$$= \frac{\left(\frac{\pi}{4}\right)^{\frac{p}{2}}}{\Gamma(p/2+1)} \to 0 \quad \text{as } p \to \infty.$$

□

3.1.4 In high dimensions, all distance are similar

Let $X \sim N_p(0, I)$. Then

$$E(||X||^2) = p, \quad \mathrm{Var}(||X||^2) = 2p$$

since $||X||^2 \sim \chi^2(p)$.
Informally, we can write $||X||^2 \approx p \pm \sqrt{2p}$. Then one can show

$$E(||X|| - \sqrt{p}) \leq \frac{1}{\sqrt{p}}, \quad \mathrm{Var}(||X||) \leq 2$$

and

$$P(||X|| - \sqrt{P} \geq \varepsilon) \leq 2e^{-c\varepsilon^2} \qquad (\forall \varepsilon \in [0, \sqrt{p}]$$

that is, $||X|| \to \sqrt{p}$ with bounded spread e.g. when $p \geq 100$ then $P(||X|| \in \sqrt{p} \pm 10) \approx 0.99$.
For $X, Y \overset{iid}{\sim} N_p(0, I)$,

$$E(||X - Y||^2) = 2p, \quad V(||X - Y||^2) = 8p$$

since $X - Y \sim N_p(0, 2I) \equiv \sqrt{2}N_p(0, I)$. Therefore $||X - Y||^2 \sim 2\chi^2(p)$.
We can also show

$$E(||X - Y|| - \sqrt{2p}) \leq \frac{1}{\sqrt{2p}}, \quad \mathrm{Var}(||X||) \leq 3,$$

that is $||X - Y|| \to \sqrt{2p}$ with bounded spread.

3.1.5 In high dimensions, all vectors are roughly orthogonal

For $X \sim N_p(0, I)$ and $v \in \mathbb{R}^p$,

$$E(\langle X, v\rangle) = E(v^\top X) = v^\top E(X) = 0, \quad \mathrm{Var}(\langle X, v\rangle) = v^\top \mathrm{Var}(X) v = ||v||^2$$

Thus, $\langle X, v\rangle \approx 0 \pm ||v||$. Combining this result with $||X||^2 \approx p \pm \sqrt{2p}$, we get

$$\cos\langle(X, v) = \frac{\langle X, v\rangle}{||X|| \cdot ||v||} \approx \frac{0 \pm ||v||}{\sqrt{p \pm \sqrt{2p}} \cdot ||v||} \to 0$$

as $p \to \infty$.
For $X, Y \overset{iid}{\sim} N_p(0, I)$, $E(\langle X, Y\rangle) = 0$ and $\mathrm{Var}(\langle X, Y\rangle) = p$.

Proof.

$$E\left(\sum_{i=1}^{p} X_i Y_i\right) = \sum_{i=1}^{p} E(X_i)E(Y_i) = 0,$$

and

$$\mathrm{Var}\left(\sum_{i=1}^{p} X_i Y_i\right) = \sum_{i=1}^{p} E(X_i^2 Y_i^2) = \sum_{i=1}^{p} E(X_i^2)E(Y_i^2) = p.$$

□

Thus, $\langle X, Y\rangle \approx 0 \pm \sqrt{p}$. Combining this with $||X||^2, ||Y||^2 \approx p \pm \sqrt{2p}$, we get

$$\cos\langle(X, Y) = \frac{\langle X, Y\rangle}{||X|| \cdot ||Y||} \approx \frac{0 \pm \sqrt{p}}{p \pm \sqrt{2p}} \to 0$$

as $p \to \infty$. One can also show

$$P\left(\left|\frac{\langle X, Y\rangle}{||X|| \cdot ||Y||} \geq \varepsilon\right|\right) \leq \frac{2/\varepsilon + 7}{\sqrt{p}}, \qquad (\forall \varepsilon > 0).$$

Remark. $\cos\langle(X, Y) = \langle X/||X||, Y/||Y||, =\rangle\langle\tilde{X}, \tilde{Y}\rangle$. Then since $X/||X|| \sim \mathrm{Unif}(S_{p-1}(1))$ we get that $f_X(x) \propto \exp(-0.5||X||^2)$ which depends only on the distance from the origin, not the direction.
This tells us that result 5 is equivalent to the fact that two random vectors on a unit sphere are roughly orthogonal to each other.

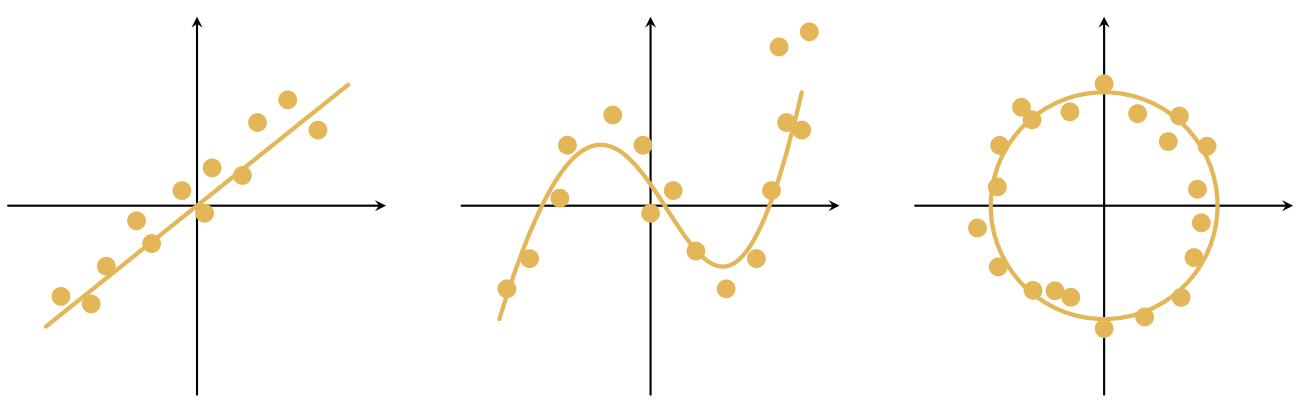
Chapter 4

Principal Component Analysis (PCA)

4.1 Dimension Reduction

We start with the assumption that our data lie on a low-dimensional manifold.

Example. $x_1, \ldots, x_n \in \mathbb{R}^2$,



PCA assumes the manifold is a r-dimensional affine subspace, e.g. a hyper-plane.

4.2 PCA

To find an r-dimensional place, we need a point $a \in \mathbb{R}^p$, a basis $v_1, \ldots, v_r \in \mathbb{R}^p$.

4.2.1 Maximal Variance view

Assume that

$$X = \begin{pmatrix} — x_1^\top — \\ \vdots \\ — x_n^\top — \end{pmatrix} \in \mathbb{R}^{n \times p}$$

is *column-centered*:

$$\overline{x} = 0, \qquad S = \frac{X^\top X}{n}.$$

A **good** direction v will maximize the spread of the projections $z_1, \ldots, z_n$.
The projection of x_i onto v is

$$z_i = x_i^\top v, \qquad \hat{x}_i = Pv = vv^\top x_i = z_i \cdot v.$$

In matrix form,

$$z = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} = Xv, \qquad \overline{z} = 0, \qquad S_z^2 = \frac{z^\top z}{n} = v^\top S v.$$

Optimization problem:

$$\max_{v \in \mathbb{R}^p} v^\top S v \quad \text{subject to } \|v\| = 1.$$

4.2.2 Detour: Method of Lagrange Multipliers

Consider $x \in \mathbb{R}^p$ and two functions $f : \mathbb{R}^p \to \mathbb{R}$ and $g : \mathbb{R}^p \to \mathbb{R}$.
We want to maximize $f(x)$ subject to $g(x) = 0$.

1. Form the **Lagrangian**:
$$\mathcal{L}(x, \lambda) = f(x) + \lambda \cdot g(x),$$
where λ is the *Lagrange multiplier*.

2. Find critical points of $\mathcal{L}$:
$$\nabla \mathcal{L}(x, \lambda) = 0.$$

3. Among all critical points (x^*, λ^*) of $\mathcal{L}$, find the one that results in the largest $f(x^*)$.

Geometric interpretation: At the optimum x^*, the contours of $f(x)$ and $g(x)$ are tangent, i.e., $\nabla f(x^*)$ and $\nabla g(x^*)$ are parallel. There exists λ^* such that $\nabla f(x^*) = \lambda^* \nabla g(x^*)$.

Multiple constraints.

$$\max f(x) \quad \text{subject to } g_1(x) = 0, \ \ldots, \ g_k(x) = 0.$$

1. Form the Lagrangian:
$$\mathcal{L}(x, \lambda_1, \ldots, \lambda_k) = f(x) + \lambda_1 g_1(x) + \cdots + \lambda_k g_k(x).$$

2. Solve $\nabla \mathcal{L}(x, \lambda_1, \ldots, \lambda_k) = 0$ to find critical points $x^*, \lambda_1^*, \ldots, \lambda_k^*$.

3. Use x^* with the largest value of $f(x^*)$.

4.2.3 PC1

Proposition (PC1). $v = v_1$ is the **top eigenvector** of S, and the corresponding $S_z^2 = \lambda_1$ is the **top eigenvalue** of S.

Proof. Apply Lagrange multipliers to

$$\max\ v^\top S v \quad \text{subject to } \|v\|^2 = 1.$$

Form the Lagrangian:

$$\mathcal{L}(v, \lambda) = v^\top S v - \lambda(v^\top v - 1).$$

Differentiate and set to zero:

$$\nabla_v \mathcal{L} = 2Sv - 2\lambda v = 0 \implies Sv = \lambda v,$$

so v is an eigenvector of S. Then

$$\mathrm{Var}(z) = v^\top S v = \lambda \|v\|^2 = \lambda \xrightarrow{\max} \lambda_{\max}.$$

Thus v corresponds to the maximum value, i.e., $v = v_1$ is the top eigenvector of S. □

4.2.4 Terminology

- $v_1 \in \mathbb{R}^p$ is the **PC direction**.
- $z = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} = X v_1$ is the **score vector**.
- $\lambda_1 = v_1^\top S v_1$ is the **variance explained**.
- $\hat{X}_1 = \begin{pmatrix} \hat{x}_1^\top \\ \vdots \\ \hat{x}_n^\top \end{pmatrix} = X v_1 v_1^\top$ are the **projections onto the PC$_1$ space**.

4.2.5 PC2

Find a new direction v with $\|v\| = 1$ that maximizes S_z^2 and is perpendicular to v_1:

$$\max_{v \in \mathbb{R}^p}\ v^\top S v \quad \text{subject to } \|v\| = 1 \text{ and } v \perp v_1.$$

Proposition (PC2). $v = v_2$ is the **second largest eigenvector** of S, where $S_z^2 = \lambda_2$ is the second eigenvalue of S.

Proof. Form the Lagrangian:

$$\mathcal{L}(v, \lambda, \mu) = v^\top S v - \lambda(v^\top v - 1) - \mu v^\top v_1.$$

Differentiate and set to zero:

$$\nabla_v \mathcal{L} = 2Sv - 2\lambda v - \mu v_1 = 0.$$

Multiply on the left by $v_1^\top$:

$$2v_1^\top S v - 2\lambda \underbrace{v_1^\top v}_{=0} - \mu \underbrace{v_1^\top v_1}_{=1} = 0 \implies \mu = 2v_1^\top S v.$$

Since $Sv_1 = \lambda_1 v_1$, we have $v_1^\top S v = v^\top S v_1 = \lambda_1 v^\top v_1 = 0$, so $\mu = 0$. Thus $Sv = \lambda v$, and v is an eigenvector of S. Both $Sv = \lambda v$ and $v \perp v_1$ are satisfied by $v = v_2$.
Therefore, v corresponds to the second largest eigenvalue. □

The second PC has direction v_2, score $z_2 = Xv_2$, and explains $\lambda_2 = v_2^\top S v_2$ variance of the data.

4.2.6 $(\mathbf{PC}_1, \mathbf{PC}_2)$-Space

The directions are $V = (v_1, v_2) \in \mathbb{R}^{p \times 2}$, the projections are $\hat{X} = Xv_1v_1^\top + Xv_2v_2^\top$, and the scores are $Z = (z_1, z_2) = X(v_1, v_2)$.

4.2.7 General PCr

$$\max_{v \in \mathbb{R}^p} v^\top S v \quad \text{subject to } \|v\| = 1 \text{ and } v \perp v_1, \ldots, v_{r-1}.$$

The solution is $v = v_r$, the r-th largest eigenvector of S, where λ_r is the r-th eigenvalue of S.

The $(\mathrm{PC}_1, \ldots, \mathrm{PC}_r)$-space:

- **Directions:** $V_{p \times r} = (v_1, v_2, \ldots, v_r) \in \mathbb{R}^{p \times r}$.
- **Scores:** $Z = (z_1, z_2, \ldots, z_r) = X \cdot V_{p \times r} \in \mathbb{R}^{n \times r}$.
- **Projections:** $\hat{X} = \begin{pmatrix} \hat{x}_1^\top \\ \vdots \\ \hat{x}_n^\top \end{pmatrix} = XV_{p \times r}V_{p \times r}^\top \in \mathbb{R}^{n \times p}$.

4.2.8 PCA: Population Version

Let $X = \begin{pmatrix} X_1 \\ \vdots \\ X_p \end{pmatrix}$ be a random vector.

- **PC1:** Find direction $v \in \mathbb{R}^p$ with $\|v\| = 1$ such that $z = X^\top v$ has maximum variance.
- **PCr:** Find direction $v \in \mathbb{R}^p$ with $\|v\| = 1$ such that $z = X^\top v$ has maximum variance and $v \perp v_1, \ldots, v_{r-1}$.

Since $\mathrm{Var}(z) = \mathrm{Var}(X^\top v) = v^\top \Sigma v$ where $\Sigma = \mathrm{Var}(X)$, the directions $v_1, \ldots, v_r$ are the eigenvectors of Σ, and

$$\mathrm{Var}(X^\top v_i) = v_i^\top \Sigma v_i = \lambda_i \|v_i\|^2 = \lambda_i.$$

4.2.9 Biplot (Visualization in 2D)

Example. $X = (X_{\text{height}}, X_{\text{weight}}, X_{\text{age}})^\top \in \mathbb{R}^3$.
The original variables correspond to the standard basis vectors:

$$\begin{aligned} \text{height} &: (1,0,0)^\top, \\ \text{weight} &: (0,1,0)^\top, \\ \text{age} &: (0,0,1)^\top. \end{aligned}$$

Let $V = (v_1, v_2)$ be the first two PC directions. The scores are

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = V^\top X = (v_1, v_2)^\top X = \begin{pmatrix} v_1^\top X \\ v_2^\top X \end{pmatrix},$$

so the data points are plotted as (z_{i1}, z_{i2}) for $i = 1, \ldots, n$ in the 2D score space. Each original variable e_j is displayed as the vector $(v_1^\top e_j,\ v_2^\top e_j)$.

4.3 Practical Aspects of PCA

4.3.1 PCA is Sensitive to Scaling

If the variables are measured in different units (e.g., weight in kg vs. height in cm vs. height in m), PCA will be dominated by the variable with the largest variance. When necessary, **standardize** the data columns before applying PCA:

$$x_{ij} \leftarrow \frac{x_{ij} - \overline{x}_j}{\hat{\sigma}_j},$$

so that each column has mean 0 and variance 1.

4.3.2 How to Pick r?

- For **visualization**, pick $r = 2$ or $r = 3$.
- For **data compression** or **denoising**, compute the *proportion of variance explained* (PVE).

The total variance in the data is $\lambda_1 + \cdots + \lambda_p$.

Variance explained (VE) by PC_j:	λ_j
Proportion of variance explained (PVE) by PC_j:	$\dfrac{\lambda_j}{\lambda_1 + \cdots + \lambda_p}$

Scree Plot

Plot PVE against the PC index. Two common strategies:

- **Threshold rule:** Find the smallest r such that $\mathrm{PC}_1, \ldots, \mathrm{PC}_r$ explain $\geq 90\%$ of the variance. For example, with PVE values $50\%, 30\%, 15\%, 2\%, 1\%, 1\%, 1\%, \ldots$, one would choose $r = 3$ (cumulative PVE $= 95\%$).
- **Elbow method:** Remove PCs with a "small" PVE value. Identify the elbow in the scree plot and discard all PCs to the right (the "unimportant PCs").

4.3.3 PCA Complexity

Input: $X \in \mathbb{R}^{n \times p}$.

1. **Column-centering:** Compute $\overline{x}$ and set $x_i \leftarrow x_i - \overline{x}$ for all $i = 1, \ldots, n$. $O(np)$
2. **Compute** $S = \dfrac{X^\top X}{n}$. $O(np^2)$
3. **Compute eigendecomposition** of $S \in \mathbb{R}^{p \times p}$. $O(p^3)$

Output: Directions $v_1, \ldots, v_p$ and variances $\lambda_1, \ldots, \lambda_p$.

Total complexity: $O(np^2 + p^3)$.

Often, we need only r eigenvectors of $S \in \mathbb{R}^{p \times p}$.

4.4 Power Iteration

Goal: Find λ_1 of $S \in \mathbb{R}^{p \times p}$, assuming $\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_p > 0$.

Input: $S \in \mathbb{R}^{p \times p}$ and an arbitrary $w \in \mathbb{R}^p$.

1. $y = Sw$
2. $v = \dfrac{y}{\|y\|}$
3. Repeat until convergence: $\|v^{(t)} - v^{(t-1)}\| \leq \delta$.

Output: $\hat{v}_1$.

Why it works. Let $S = U\Lambda U^\top = \sum_{i=1}^{p} \lambda_i u_i u_i^\top$ be the eigendecomposition. Then

$$S^k w = \sum_{i=1}^{p} \lambda_i^k (u_i^\top w)\, u_i = \lambda_1^k \left[(u_1^\top w)\, u_1 + \sum_{i=2}^{p} \left(\frac{\lambda_i}{\lambda_1}\right)^k (u_i^\top w)\, u_i \right] \xrightarrow{k\to\infty} \lambda_1^k (u_1^\top w)\, u_1,$$

since $(\lambda_i/\lambda_1)^k \to 0$ for $i \geq 2$.

4.4.1 PCA via Power Iteration

1. Use power iteration to find v_1 of S; set $\lambda_1 = v_1^\top S v_1$.
2. **Deflate:** Let $\tilde{X} = X(I - v_1 v_1^\top)$. Then
$$\tilde{X}^\top \tilde{X} = X^\top X - X^\top X v_1 v_1^\top.$$
3. Compute the deflated sample covariance:
$$\tilde{S} = \frac{1}{n} \tilde{X}^\top \tilde{X} = S - \lambda_1 v_1 v_1^\top.$$
Equivalently,
$$\tilde{S} = (I - v_1 v_1^\top) S (I - v_1 v_1^\top) = S - \lambda_1 v_1 v_1^\top - \lambda_1 v_1 v_1^\top + \lambda_1 v_1 v_1^\top = S - \lambda_1 v_1 v_1^\top.$$
4. Use power iteration on $\tilde{S}$ to find v_2 and $\lambda_2 = v_2^\top S v_2$.

4.5 PCA: Minimum Reconstruction Error View

Goal: Find an r-dimensional plane that approximates $x_1, \ldots, x_n \in \mathbb{R}^p$ "well". We need:

- a point $a \in \mathbb{R}^p$,
- an orthogonal basis $v_1, \ldots, v_r \in \mathbb{R}^p$.

Any point $\tilde{x}$ on the plane is

$$\tilde{x} = a + z_1 v_1 + \cdots + z_r v_r \quad \text{for some } z_1, \ldots, z_r \in \mathbb{R}.$$

In particular, writing $V = (v_1, \ldots, v_r) \in \mathbb{R}^{p\times r}$ and $\beta_i = \begin{pmatrix} z_{i1} \\ \vdots \\ z_{ir} \end{pmatrix} \in \mathbb{R}^r$ with $V^\top V = I_r$:

$$\tilde{x}_i = a + V\beta_i.$$

The PCs minimize the reconstruction error:

$$\mathrm{RE}(a, V, \beta_1, \ldots, \beta_n) = \sum_{i=1}^{n} \|x_i - \tilde{x}_i\|^2 = \sum_{i=1}^{n} \|x_i - a - V\beta_i\|^2$$

with respect to $a \in \mathbb{R}^p$, $V \in \mathbb{R}^{p\times r}$, $\beta_1, \ldots, \beta_n \in \mathbb{R}^r$, subject to $V^\top V = I_r$.

4.5.1 Optimizing over β_i

For fixed V and a, the optimal β_i is

$$\beta_i^* = V^\top (x_i - a).$$

This is simply the least-squares projection: $\hat{\beta}_i = (V^\top V)^{-1} V^\top (x_i - a) = V^\top (x_i - a)$.

4.5.2 Optimizing over a

Substituting β_i^*:

$$\mathrm{RE}(a, V) = \sum_{i=1}^{n} \|(I - VV^\top)(x_i - a)\|^2.$$

This is minimized at $a = c$ for any $c \in \mathbb{R}^p$ satisfying $(I - VV^\top)(\overline{x} - c) = 0$, i.e., $\overline{x} - c \in \mathrm{col}(V)$. Setting $\nabla_a \mathrm{RE} = 0$:

$$-2 \sum_{i=1}^{n} (I - VV^\top)(x_i - a) = -2n(I - VV^\top)(\overline{x} - a) = 0 \implies \overline{x} - a \in \mathrm{col}(V).$$

In particular, $a = \overline{x}$ is optimal. For centered data ($\overline{x} = 0$), $a = 0$.

4.5.3 Optimizing over V

After substituting $a = \overline{x}$ and $\beta_i^* = V^\top (x_i - \overline{x})$:

$$\mathrm{RE}(V) = \sum_{i=1}^{n} \|(I - VV^\top)(x_i - \overline{x})\|^2 = (n-1)\,\mathrm{tr}\big((I - VV^\top)S\big)\,,$$

which is minimized at $V_{(r)} = (v_1, \ldots, v_r)$, where $v_1, \ldots, v_r$ are the top eigenvectors of S.

Proof sketch.

$$\begin{aligned}
\mathrm{RE} &= \sum_{i=1}^{n} \mathrm{tr}\big((x_i - \overline{x})^\top (I - VV^\top)(x_i - \overline{x})\big) \\
&= \mathrm{tr}\left((I - VV^\top) \sum_{i=1}^{n} (x_i - \overline{x})(x_i - \overline{x})^\top \right) \\
&= (n-1)\,\mathrm{tr}\big((I - VV^\top)S\big) \\
&= (n-1)\Big(\mathrm{tr}(S) - \mathrm{tr}(V^\top S V)\Big).
\end{aligned}$$

Minimizing RE is equivalent to

$$\max_{V \in \mathbb{R}^{p \times r}} \mathrm{tr}(V^\top S V) \quad \text{subject to } V^\top V = I_r,$$

whose solution is $V = (v_1, \ldots, v_r)$, the top r eigenvectors of S.

For centered data, the projection of x_i onto the r-dimensional plane is

$$\hat{x}_i = VV^\top x_i, \qquad \hat{X} = XV_{(r)}V_{(r)}^\top.$$

Interpretation (denoising):

$$x_i = \underbrace{\hat{x}_i}_{\text{true, denoised signal}} + \underbrace{(x_i - \hat{x}_i)}_{\text{noise}}.$$

4.5.4 Example: Imaging Data

Each image is a 16×16 pixel image of the digit "3", flattened to a vector $x_i \in \mathbb{R}^{256}$.

$$X = \begin{pmatrix} — x_1^\top — \\ \vdots \\ — x_n^\top — \end{pmatrix}, \qquad n = \#\text{ of images}, \quad p = 256\text{pixels}.$$

The data live in a 256-dimensional space, but PCA finds a 2-dimensional plane that captures most of the variation. Any point x is approximated by

$$\hat{x} = \overline{x} + z_1 \cdot v_1 + z_2 \cdot v_2,$$

where v_1 captures "digit width" and v_2 captures "line thickness".

4.5.5 Example: Functional Data

Each observation is the knee flexion angle curve $x_i : [0, 100] \to \mathbb{R}$ over a gait cycle, discretized at $p = 100$ time points.

$$X = \begin{pmatrix} — x_1^\top — \\ \vdots \\ — x_n^\top — \end{pmatrix}, \qquad n = \#\text{ people}, \quad p = 100\text{timepoints}.$$

Any point x is approximated by

$$\hat{x} = \overline{x} + z_1 \cdot v_1 + z_2 \cdot v_2,$$

where v_1 ("size") captures vertical shift and v_2 ("shape") captures phase shift. The black thick line in the plot is $\overline{x}$.

4.6 PCA via SVD

For centered $X \in \mathbb{R}^{n \times p}$ with SVD $X = UDV^\top$ (where $U \in \mathbb{R}^{n \times n}$, $D = \text{diag}(d_1, \ldots, d_p)$, $V \in \mathbb{R}^{p \times p}$):

$$S = \frac{X^\top X}{n} = \frac{VD^\top U^\top UDV^\top}{n} = \frac{VD^2V^\top}{n}.$$

- $v_1, \ldots, v_p$ are the **right singular vectors** of $X \Rightarrow$ PC directions.
- $\lambda_i = d_i^2/n$ are the **variances explained**.
- $z_i = U_{(r)} D_{(r)}$ are the **scores**, where

$$Z_{(r)} = XV_{(r)} = UDV^\top V_{(r)} = U_{(r)} D_{(r)}.$$

- $\hat{X}_{(r)} = U_{(r)} D_{(r)} V_{(r)}^\top$ are the **projections**.

Interpretation. The full SVD gives

$$X = d_1 u_1 v_1^\top + d_2 u_2 v_2^\top + \cdots + d_p u_p v_p^\top,$$

where the first terms capture the signal and the remaining terms capture noise.

4.7 PCA: Low-Rank Matrix Approximation View

If $\tilde{x}_1, \ldots, \tilde{x}_n$ belong to an r-dimensional plane, then

$$\tilde{x}_i = V\beta_i, \quad \tilde{X} = \begin{pmatrix} \tilde{x}_1^\top \\ \vdots \\ \tilde{x}_n^\top \end{pmatrix} = ZV^\top, \quad Z \in \mathbb{R}^{n\times r},\ V \in \mathbb{R}^{p\times r},$$

so $\operatorname{rank}(\tilde{X}) \le r$.
Low-Rank Matrix Approximation (LRMA) problem:

$$\min_{\tilde{X}\in\mathbb{R}^{n\times p}} \|X - \tilde{X}\|_F^2 \quad \text{subject to } \operatorname{rank}(\tilde{X}) \le r,$$

where the reconstruction error is

$$\|X - \tilde{X}\|_F^2 = \sum_{i=1}^n \sum_{j=1}^p (x_{ij} - \tilde{x}_{ij})^2 = \sum_{i=1}^n \|x_i - \tilde{x}_i\|^2.$$

4.7.1 Reformulation

Let $\tilde{X} = QA$ for column-orthogonal $Q \in \mathbb{R}^{n\times r}$. The LRMA problem becomes:

$$\min_{Q\in\mathbb{R}^{n\times r},\ A\in\mathbb{R}^{r\times p}} \|X - QA\|_F^2 \quad \text{subject to } Q^\top Q = I_r.$$

If Q is fixed: by ordinary regression,

$$A^* = (Q^\top Q)^{-1} Q^\top X = Q^\top X.$$

4.7.2 Reducing to an Optimization over Q

Let $X = UDV^\top$ be the SVD. Then

$$\|X - QQ^\top X\|_F^2 = \|(I - QQ^\top)X\|_F^2 = \mathrm{tr}\big(X^\top(I - QQ^\top)X\big) = \mathrm{tr}(X^\top X) - \mathrm{tr}(Q^\top XX^\top Q).$$

Minimizing this is equivalent to:

$$\max_{Q \in \mathbb{R}^{n\times r}} \ \mathrm{tr}(Q^\top XX^\top Q) \quad \text{subject to } Q^\top Q = I_r.$$

Note $XX^\top = UD^2U^\top$. Setting $H = U^\top QU$ (an $n \times n$ matrix):

$$\mathrm{tr}(Q^\top XX^\top Q) = \mathrm{tr}(Q^\top UD^2U^\top Q) = \mathrm{tr}(H^\top D^2 H) = \sum_{i=1}^{n} d_i^2 h_{ii}^2 \ \le \ \sum_{i=1}^{r} d_i^2,$$

where $h_{ii} \in [0, 1]$ and the maximum is achieved when $h_{11} = \cdots = h_{rr} = 1$ and all other entries of H are zero, i.e., H is the projection onto the top-r coordinates.

4.7.3 Optimal Solution

- **Optimal H:**

$$H^* = \begin{pmatrix} I_r & 0 \\ 0 & 0 \end{pmatrix}, \qquad \mathrm{tr}(H^{*\top} D^2 H^*) = \sum_{i=1}^{r} d_i^2.$$

- **Optimal Q:**

$$H = U^\top Q \implies Q^* = UH^* = U \begin{pmatrix} I_r \\ 0 \end{pmatrix} = U_{(r)}.$$

- **Optimal A:**

$$A^* = Q^{*\top} X = U_{(r)}^\top UDV^\top = \begin{pmatrix} I_r & 0 \end{pmatrix} DV^\top = D_{(r)} V_{(r)}^\top.$$

- **Optimal $\tilde{X}$:**

$$\tilde{X}^* = Q^* A^* = U_{(r)} D_{(r)} V_{(r)}^\top.$$

This is exactly the **truncated SVD** of X to rank r, confirming that PCA solves the low-rank matrix approximation problem.

Reference: *Principal component analysis* by M. Greenacre, P. Groenen, T. Hastie, A. d'Enza, A. Markos, and E. Tuzhilina. Published in *Nature Reviews Methods Primers*, 2022.

4.8 Low-Rank Matrix Approximation (LRMA)

$$\min_{\tilde{X} \in \mathbb{R}^{n\times p}} \|X - \tilde{X}\|_F^2 \quad \text{subject to } \mathrm{rank}(\tilde{X}) \le r.$$

Here $x_1, \ldots, x_n$ are the observed data and $\tilde{x}_1, \ldots, \tilde{x}_n$ are the approximations belonging to an r-dimensional plane. The solution is

$$\tilde{X}^* = \mathrm{SVD}_r(X) = U_{(r)} D_{(r)} V_{(r)}^\top = Z_{(r)} V_{(r)}^\top,$$

where $Z_{(r)} \in \mathbb{R}^{n\times r}$ and $V_{(r)} \in \mathbb{R}^{p\times r}$.

4.8.1 Applications of Low-Rank Approximation

1. **Data compression.** Store Z and V instead of X. For rank-r approximation, one stores $n \cdot r + p \cdot r$ numbers instead of $n \cdot p$. As r increases, the approximation $\hat{X} = \mathrm{SVD}_r(X)$ becomes more faithful to the original X.

2. **De-noising.** The true signal is low-rank, but we observe only a noisy version X. Recover the signal via $\hat{X} = \mathrm{SVD}_r(X)$ for a suitable r.

3. **Imputation of missing values.**

   Netflix Competition: Build a recommendation system with $n = 480$k customers, $p = 17$k movies, and only 1% of values observed.

   The idea is to approximate the ratings matrix as

   $$x_i \approx z_{i1} v_1 + \cdots + z_{ir} v_r,$$

   where v_j represents a "genre" direction (e.g., thriller, romance) and z_{ij} represents how much customer i likes that genre. Schematically:

   $$X_{\text{customers}\times\text{movies}} \approx Z_{\text{customers}\times\text{cliques}} \cdot V^{\top}_{\text{cliques}\times\text{movies}}.$$

4.9 Low-Rank Matrix Completion

Assume $X \in \mathbb{R}^{n\times p}$ has missing values. Denote by $\Omega = \{(i,j) : x_{ij} \text{ observed}\}$ the set of observed entries, and by $W \in \mathbb{R}^{n\times p}$ the **mask matrix**:

$$W_{ij} = \begin{cases} 1 & \text{if } (i,j) \in \Omega, \\ 0 & \text{otherwise.} \end{cases}$$

Example.

$$X = \begin{pmatrix} 1 & \cdot \\ \cdot & 2 \end{pmatrix}, \quad \Omega = \{(1,1),(2,2)\}, \quad W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

4.9.1 Reconstruction Error with Missing Values

$$\sum_{(i,j)\in\Omega} (x_{ij} - \tilde{x}_{ij})^2 = \|W \odot (X - \tilde{X})\|_F^2,$$

where $A \odot B$ denotes the **element-wise (Hadamard) product**: $(A \odot B)_{ij} = A_{ij} B_{ij}$.

Verification:

$$\sum_{(i,j)\in\Omega} (x_{ij} - \tilde{x}_{ij})^2 = \sum_{i=1}^{n} \sum_{j=1}^{p} \big[W_{ij}(x_{ij} - \tilde{x}_{ij})\big]^2 = \|W \odot (X - \tilde{X})\|_F^2.$$

4.9.2 Hard-Impute Algorithm

Problem:

$$\min_{\tilde{X}\in\mathbb{R}^{n\times p}} \|W \odot (X - \tilde{X})\|_F^2 \quad \text{subject to } \operatorname{rank}(\tilde{X}) \le r.$$

Input: Matrix $X \in \mathbb{R}^{n\times p}$ and rank r.
Initialization: $\tilde{X} \in \mathbb{R}^{n\times p}$ (e.g., zeros).
Repeat until convergence:

1. $Y = W \odot X + (1 - W) \odot \tilde{X}$ (impute missing values with current $\tilde{X}$)
2. $Y = UDV^\top$ (compute SVD of Y)
3. $\tilde{X} = U_{(r)}D_{(r)}V_{(r)}^\top$ (project onto rank-r matrices)

Output: $\tilde{X} \in \mathbb{R}^{n\times p}$.

The two steps are:

1. Impute missing values in X with the elements from $\tilde{X}$.
2. Make $\tilde{X}$ low-rank.

Example. Let $X = \begin{pmatrix} 1 & \cdot \\ \cdot & 2 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $\tilde{X}^{(0)} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$.

Step 1: $Y = W \odot X + (1 - W) \odot \tilde{X} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$.
Step 2: $\tilde{X} = U_{(r)}D_{(r)}V_{(r)}^\top$.

Computational trick. If there are many missing values and r is small, use the update

$$Y = -W \odot (X - \tilde{X}) + \tilde{X},$$

storing only $(x_{ij} - \tilde{x}_{ij})$ for $(i, j) \in \Omega$ (sparse) and only $U_{(r)}, D_{(r)}, V_{(r)}$ (low-rank).

4.10 Soft Low-Rank Approximation

4.10.1 Hard vs. Soft Constraint

1. **Hard constraint:**

$$\min_{\tilde{X}\in\mathbb{R}^{n\times p}} \|X - \tilde{X}\|_F^2 \quad \text{subject to } \operatorname{rank}(\tilde{X}) \le r.$$

   Controlling $\operatorname{rank}(\tilde{X}) \le r$ is equivalent to controlling the number of non-zero singular values (i.e., $d_i > 0$ for at most r indices).

2. **Soft (penalized) formulation.** Instead of constraining $\sum_i \mathbf{1}[d_i > 0] \leq r$, we control $\sum_i d_i$, i.e., the **nuclear norm** $\|\tilde{X}\|_* = \sum_i d_i(\tilde{X})$:

$$\min_{\tilde{X} \in \mathbb{R}^{n \times p}} \|X - \tilde{X}\|_F^2 + \delta \|\tilde{X}\|_*.$$

The penalty δ balances approximation accuracy and the rank of $\tilde{X}$. If $\delta \to 0$, then $\operatorname{rank}(\tilde{X}) = p$ (i.e., $\tilde{X} = X$).

4.10.2 Solutions

Hard: $\operatorname{rank}(\tilde{X}) \leq r$	**Soft:** $\|\tilde{X}\|_*$ penalty
Solution: $\tilde{X} = \mathrm{SVD}_r(X)$	Solution: $\tilde{X} = S_\delta(X)$
$\tilde{X} = U\tilde{D}V^\top$	$\tilde{X} = U\tilde{D}V^\top$
$\tilde{D} = \operatorname{diag}(d_1, \ldots, d_r, 0, \ldots, 0)$	$\tilde{D} = \operatorname{diag}\big((d_1 - \delta)_+, \ldots, (d_p - \delta)_+\big)$

Here $(d - \delta)_+ = \max(d - \delta, 0)$ is the **soft-thresholding** operator.

4.10.3 Soft-Impute Algorithm

Problem:

$$\min_{\tilde{X} \in \mathbb{R}^{n \times p}} \|W \odot (X - \tilde{X})\|_F^2 + \delta \|\tilde{X}\|_*.$$

Input: Matrix $X \in \mathbb{R}^{n \times p}$ and penalty δ.
Initialization: $\tilde{X} \in \mathbb{R}^{n \times p}$.
Repeat until convergence:

1. $Y = W \odot X + (1 - W) \odot \tilde{X}$
2. $\tilde{X} = S_\delta(Y)$

Output: $\tilde{X} \in \mathbb{R}^{n \times p}$.

The Netflix test error comparison (RMSE vs. rank) shows that **SoftImpute** outperforms **HardImpute** at higher ranks and beats Cinematch (Netflix's in-house algorithm) for sufficiently large rank.

4.11 Proximal Operators

Definition. Given $h : \mathbb{R}^n \to \mathbb{R}$, the **proximal mapping** is

$$\operatorname{prox}_\delta(a) = \operatorname*{argmin}_{x \in \mathbb{R}^n} \left\{ \frac{1}{2}\|a - x\|^2 + \delta h(x) \right\}.$$

Example (Projection onto a convex set). Let $h(x) = \mathbf{1}_C(x) = \begin{cases} 0 & x \in C \\ +\infty & x \notin C \end{cases}$.

$$\operatorname{prox}_\delta(a) = \operatorname*{argmin}_{x\in\mathbb{R}^n} \frac{1}{2}\|a-x\|^2 + \delta \mathbf{1}_C(x) = \operatorname*{argmin}_{x\in C} \|a-x\|^2 = \operatorname{proj}_C(a).$$

Example (Projection onto rank-r matrices). Let $C = \{X \in \mathbb{R}^{n\times p} : \operatorname{rank}(X) \le r\}$.

$$\operatorname{prox}_\delta(A) = \operatorname*{argmin}_{X\in C} \|A-X\|_F^2 = \operatorname{proj}_C(A) = \operatorname{SVD}_r(A).$$

Example (Soft-thresholding on $\mathbb{R}$). Let $x \in \mathbb{R}$ and $h(x) = |x|$. Then

$$\operatorname{prox}_\delta(a) = \operatorname*{argmin}_{x\in\mathbb{R}} \left\{ \frac{1}{2}(a-x)^2 + \delta|x| \right\} = S_\delta(a) = \operatorname{sign}(a)(|a|-\delta)_+.$$

This is the **soft-thresholding operator**. Note:

- $S_\delta(a) = a$ for $\delta = 0$.
- $S_\delta(\alpha a) = \alpha S_{\alpha\delta}(a)$ for $\alpha > 0$ (scaling property).
- If $a > 0$ and $|a| > \delta$: $S_\delta(a) = a - \delta$.
- If $|a| \le \delta$: $|a| - \delta \le 0$, so $S_\delta(a) = 0$.
- If $a < 0$ and $|a| > \delta$: $S_\delta(a) = -(|a|-\delta) = a + \delta$.

Example (Soft-thresholding on $\mathbb{R}^n$). Let $x \in \mathbb{R}^n$ and $h(x) = \|x\|_1 = \sum_i |x_i|$.

$$\operatorname{prox}_\delta(a) = \operatorname*{argmin}_{x\in\mathbb{R}^n} \left\{ \frac{1}{2}\|a-x\|^2 + \delta\|x\|_1 \right\} = S_\delta(a),$$

where S_δ is applied coordinate-wise. This follows from

$$\frac{1}{2}\|a-x\|^2 + \delta\|x\|_1 = \sum_i \left[\frac{1}{2}(a_i - x_i)^2 + \delta|x_i| \right].$$

Example (Soft-thresholding on matrices (nuclear norm)). Let $X \in \mathbb{R}^{n\times p}$ and $h(X) = \|X\|_* = \sum_i d_i(X)$.

$$\operatorname{prox}_\delta(A) = \operatorname*{argmin}_{X\in\mathbb{R}^{n\times p}} \left\{ \|A-X\|_F^2 + \delta\|X\|_* \right\} = S_\delta(A),$$

where for $A = UDV^\top$, $S_\delta(A) = U \operatorname{diag}((d_i - \delta)_+) V^\top$. This follows from the von Neumann trace inequality (HW3).

4.12 Proximal Gradient Descent

Problem:

$$\min_{x\in\mathbb{R}^n} \; f(x) + h(x),$$

where $f(x)$ is convex and differentiable, and $h(x)$ is convex but possibly not differentiable. **Proximal gradient descent** repeats until convergence:

1. $y = x - \alpha\nabla f(x)$ (gradient step)
2. $x = \mathrm{prox}_\alpha(y) = \operatorname*{argmin}_{x\in\mathbb{R}^n}\left\{\frac{1}{2}\|y-x\|^2 + \alpha h(x)\right\}$ (proximal step)

Special case: constrained optimization. For

$$\min_{x\in C} f(x) \equiv \min_{x\in\mathbb{R}^n} f(x) + \mathbf{1}_C(x),$$

the proximal step reduces to $x = \mathrm{prox}_\alpha(y) = \mathrm{proj}_C(y)$.

4.12.1 Proximal Gradient Descent Applied to Matrix Completion

Example 1 (Hard-impute).

$$\min_{\tilde{X}\in\mathbb{R}^{n\times p}} \|W\odot(X-\tilde{X})\|_F^2 \quad \text{subject to } \mathrm{rank}(\tilde{X}) \le r.$$

Set $\alpha = 1$, $f(\tilde{X}) = \|W\odot(X-\tilde{X})\|_F^2$, $h(\tilde{X}) = \mathbf{1}_C(\tilde{X})$ where $C = \{\mathrm{rank} \le r\}$.

1. $Y = \tilde{X} - \nabla f(\tilde{X}) = \tilde{X} + W\odot(X-\tilde{X}) = W\odot X + (1-W)\odot\tilde{X}$
2. $\tilde{X} = \mathrm{prox}_1(Y) = \mathrm{proj}_C(Y) = \mathrm{SVD}_r(Y)$

Example 2 (Soft-impute).

$$\min_{\tilde{X}\in\mathbb{R}^{n\times p}} \|W\odot(X-\tilde{X})\|_F^2 + \delta\|\tilde{X}\|_*.$$

Set $\alpha = 1$, $f(\tilde{X}) = \|W\odot(X-\tilde{X})\|_F^2$, $h(\tilde{X}) = \delta\|\tilde{X}\|_*$.

1. $Y = \tilde{X} - \nabla f(\tilde{X}) = W\odot X + (1-W)\odot\tilde{X}$
2. $\tilde{X} = \mathrm{prox}_1(Y) = S_\delta(Y)$

4.13 Sparse PCA

The first principal component score is

$$z_1 = Xv_1 = \begin{pmatrix} f_1 & \cdots & f_p \end{pmatrix}\begin{pmatrix} v_{11} \\ \vdots \\ v_{p1}\end{pmatrix} = \sum_{j=1}^p v_{j1} f_j.$$

We want to make v_1 **sparse**: select a subset of features $f_{j_1},\ldots,f_{j_s}$ to construct z_1.

4.13.1 SCoTLASS (Jolliffe et al., 2003)

$$\max_{v \in \mathbb{R}^p} v^\top S v \quad \text{subject to } \|v\|_2 = 1, \quad \|v\|_1 \le c.$$

What is a reasonable choice for c? Since $\|v\|_2 \le \|v\|_1 \le \sqrt{p}\|v\|_2$:

- If $v = (1, 0, \ldots, 0)^\top$: $\|v\|_1 = 1$.
- If $v = \left(\frac{1}{\sqrt{p}}, \ldots, \frac{1}{\sqrt{p}}\right)^\top$: $\|v\|_1 = \sqrt{p}$.

So $c \in [1, \sqrt{p}]$.

4.13.2 Penalized Matrix Decomposition (Witten et al., 2009)

Recall the power iteration update:

$$v^{(t+1)} = \frac{S v^{(t)}}{\|S v^{(t)}\|_2}.$$

Witten et al. (2009) propose replacing this with the update

$$v^{(t+1)} = \frac{S_\delta(S v^{(t)})}{\|S_\delta(S v^{(t)})\|_2},$$

where $S_\delta(a) = \operatorname{sign}(a)(|a| - \delta)_+$ is the soft-thresholding operator applied coordinate-wise, and δ is selected such that $\|v^{(t+1)}\|_1 = c$.

4.13.3 Sparse SVD (Witten et al., 2009)

Given $X \in \mathbb{R}^{n \times p}$, find

$$\min_{d \in \mathbb{R},\, u \in \mathbb{R}^n,\, v \in \mathbb{R}^p} \|X - d u v^\top\|_F^2 \quad \text{subject to } \|u\|_2 = 1,\ \|v\|_2 = 1,\ \|u\|_1 \le c_1,\ \|v\|_1 \le c_2.$$

- If c_1 and c_2 are very large, then u and v are the singular vectors (see HW2).
- The constraint $\|u\|_1 \le 1$ and $\|v\|_1 \le 1$ alone is too restrictive: if (i^*, j^*) is the index with the largest $|x_{ij}|$, then the trivial solution $u = e_{i^*}$, $v = e_{j^*}$, $d = x_{i^* j^*}$ satisfies $\|u\|_1 = \|v\|_1 = 1$ and gives $\hat{X}(u, v) = 0$.

For fixed u and d, the subproblem in v is:

$$\max_{v \in \mathbb{R}^p} u^\top X v \quad \text{subject to } \|v\|_2 = 1,\ \|v\|_1 \le c_2.$$

Using the Lagrangian $\mathcal{L}(v, \delta, \mu) = u^\top X v - \delta(\|v\|_1 - c_2) - \frac{\mu}{2}(v^\top v - 1)$, the optimality condition gives

$$X^\top u - \delta \operatorname{sign}(v) - \mu v = 0 \implies v = \frac{1}{\mu} S_\delta(X^\top u).$$

To enforce $\|v\|_2 = 1$ we set $v = \frac{S_\delta(X^\top u)}{\|S_\delta(X^\top u)\|_2}$, and δ is chosen so that $\|v\|_1 \le c_2$.

4.13.4 Sparse PCA Algorithm

Without the constraint $\|u\|_1 \leq c_1$ (i.e., u is unrestricted):

1. $u^{(t+1)} = \dfrac{Xv^{(t)}}{\|Xv^{(t)}\|_2}$
2. $v^{(t+1)} = \dfrac{S_\delta(X^\top u^{(t+1)})}{\|S_\delta(X^\top u^{(t+1)})\|_2}$

Note that

$$S_\delta\left(\frac{X^\top Xv}{\|Xv\|_2}\right) = S_{\delta'}\left(X^\top Xv\right)/\|Xv\|_2 \quad \text{for } \delta' = \delta.$$

- If $\delta = 0$: $v^{(t+1)} = \dfrac{X^\top Xv^{(t)}}{\|X^\top Xv^{(t)}\|_2}$, i.e., standard power iteration.

To find u_2, d_2, v_2, apply the same method to the deflated matrix $\tilde{X} = X - d_1 u_1 v_1^\top$.

4.14 Application: Shape Analysis (from ESLII)

Example (Corpus callosum shape data). Each observation is the corpus callosum (CC) shape for one person, annotated with landmarks:

$$\begin{pmatrix} x_1, y_1 \\ \vdots \\ x_N, y_N \end{pmatrix} \longrightarrow (x_1, y_1, \ldots, x_N, y_N).$$

The data matrix is

$$X = \begin{pmatrix} — & \cdot & — \\ & \vdots & \\ — & \cdot & — \end{pmatrix}, \quad n = 569 \text{ elderly people.}$$

The black thick line is the mean CC shape $\overline{x}$, and the red curves show the PC loading vectors v_1, v_2. Comparing standard PCA and sparse PCA loading vectors illustrates that sparse PCA yields more localized (interpretable) deformations of the CC shape, with respect to cognitive outcomes such as walking speed and verbal fluency.

Chapter 5

Non-Linear Dimension Reduction

5.1 Kernel PCA

Idea: Transform the feature space

$$f_1, \ldots, f_p \longrightarrow \phi_1(f_1, \ldots, f_p), \ldots, \phi_q(f_1, \ldots, f_p) \quad (q > p),$$

i.e., map $x_i \in \mathbb{R}^p \mapsto \phi(x_i) \in \mathbb{R}^q$, and run PCA on $\phi(x_1), \ldots, \phi(x_n)$.

Example. $(f_1, f_2) \longrightarrow (f_1,\ f_2,\ f_1^2 + f_2^2)$.
PCA on $\phi(x_1), \ldots, \phi(x_n)$ will compute scores

$$z_{(i)} = v_{1i} f_1 + v_{2i} f_2 + v_{3i}(f_1^2 + f_2^2),$$

which are *non-linear* in f_1, f_2. In 2D, the two classes (inner circle and outer ring) are not linearly separable, but after the mapping to 3D they become separable by a hyperplane.

5.1.1 Kernel Function

Denote the transformed data by $\Phi = \begin{pmatrix} \phi(x_1)^\top \\ \vdots \\ \phi(x_n)^\top \end{pmatrix} \in \mathbb{R}^{n \times q}$.

Let $K = \Phi\Phi^\top \in \mathbb{R}^{n \times n}$ be the **inner product (kernel) matrix**, with entries

$$K_{ij} = \langle \phi(x_i), \phi(x_j) \rangle = k(x_i, x_j).$$

The function $k(\cdot, \cdot)$ is called the **kernel function**.

Example (Kernel functions and their feature maps).

- **Quadratic kernel:** $k(a, b) = (1 + \langle a, b \rangle)^2$.

  For $x_i \in \mathbb{R}^p$: $k(a, b) = 1 + 2\langle a, b \rangle + \langle a, b \rangle^2$, so $\phi(x_i) \in \mathbb{R}^{1+p+\binom{p+1}{2}}$.

- **Polynomial kernel:** $k(x, y) = (1 + \langle x, y \rangle)^q$.

- **Radial basis (Gaussian) kernel:** $k(x, y) = e^{-\|x-y\|^2/2}$.

5.1.2 Kernel PCA Algorithm

Key observation: You do not need Φ to compute PCA scores — only K.

Input: Data $X \in \mathbb{R}^{n\times p}$, kernel function $k(a,b)$.

1. Compute K with entries $K_{ij} = k(x_i, x_j)$.
2. Compute the top eigenvectors $u_1, \dots, u_r$ of K and denote $d_1, \dots, d_r$ the eigenvalues.

Output: PC scores $z_j = d_j u_j$ for $j = 1, \dots, r$.

Why it works. If Φ is centered ($\overline{\phi} = 0$), then

$$\Phi = UDV^\top \implies K = \Phi\Phi^\top = UD^2U^\top.$$

So u_j's are eigenvectors of K and d_j^2 are eigenvalues. The scores are

$$z_j = \Phi v_j = UDV^\top v_j = \sum_\ell d_\ell u_\ell (V^\top v_j)_\ell = d_j u_j.$$

If Φ is not centered, replace K by

$$\tilde{K} = CKC, \quad \text{where } C = I - \tfrac{1}{n}\mathbf{1}\mathbf{1}^\top,$$

since $\tilde{\Phi} = C\Phi$ (centering the rows) gives $\tilde{K} = \tilde{\Phi}\tilde{\Phi}^\top = C\Phi\Phi^\top C = CKC$.

5.2 Local Dimension Reduction Methods

5.2.1 t-SNE (t-Distributed Stochastic Neighbor Embedding)

Given points $x_1, \dots, x_n \in \mathbb{R}^p$:

- Compute pairwise distances $\|x_i - x_j\|_2$.
- Compute probabilities p_{ij} of selecting neighbors (i,j) (using a Gaussian distribution).

Given an embedding $z_1, \dots, z_n \in \mathbb{R}^q$ ($q \ll p$):

- Compute pairwise distances $\|z_i - z_j\|_2$.
- Compute probabilities q_{ij} of selecting neighbors (i,j) (using a t-distribution).

Find $z_1, \dots, z_n$ such that p_{ij} and q_{ij} are "similar" (minimize KL divergence).

Main parameter: *perplexity*, which balances local and global attention.

tSNE vs. PCA:

+ Non-linear, good for visualization, captures local neighbours.
− Slow, struggles with noise, less interpretable:
  - No meaning of the tSNE coordinates and distances.
  - Small distances are informative; large distances are not.
  - Cluster sizes are not informative.
  - Distances between clusters are not informative.

5.2.2 UMAP (Uniform Manifold Approximation and Projection)

Given points $x_1, \ldots, x_n \in \mathbb{R}^p$:

- Construct a weighted K-neighbour graph (using a "fuzzy simplicial complex").

Given an embedding $z_1, \ldots, z_n \in \mathbb{R}^q$ ($q \ll p$):

- Construct a weighted K-neighbour graph (using a "fuzzy simplicial complex").

Find $z_1, \ldots, z_n$ such that the graphs are "similar" (minimize cross-entropy).

Two main parameters:

- `n_neighbors`: the number of nearest neighbours.
- `min_dist`: how tightly UMAP packs neighbours.

Comparing to tSNE, UMAP is faster and better at preserving more global structure.

5.2.3 Practical Aspects

Sometimes it is better to combine PCA with tSNE or UMAP:

1. Filter the data.
2. Do PCA to reduce dimensionality and noise.
3. Plot with UMAP or tSNE, trying various hyperparameters.

5.3 Perturbation Theory for PCA

Given $S \in \mathbb{R}^{p \times p}$, consider $\tilde{S} = S + E \in \mathbb{R}^{p \times p}$, where $E \in \mathbb{R}^{p \times p}$ is a symmetric noise matrix. Let $S = U \Lambda U^\top$ and $\tilde{S} = \tilde{U} \tilde{\Lambda} \tilde{U}^\top$ be the respective eigendecompositions.
Recall $\|A\|_2 = \sqrt{\lambda_{\max}(A^\top A)} = d_1(A)$.

Theorem (Weyl's Inequality).

$$\max_{i=1,\ldots,p} |\lambda_i - \tilde{\lambda}_i| \leq \|S - \tilde{S}\|_2 = \|E\|_2,$$

i.e., eigenvalues are stable under perturbation.

Proof sketch for λ_1.

$$\lambda_1 = \max_{\|v\|=1} v^\top S v = \max_{\|v\|=1} (v^\top S v + v^\top E v) \leq \tilde{\lambda}_1 + \max_{\|v\|=1} |v^\top E v| \leq \tilde{\lambda}_1 + \|E\|_2.$$

By symmetry, $|\lambda_1 - \tilde{\lambda}_1| \leq \|E\|_2$. $\square$

5.3.1 Eigenvectors Are Not Stable

Example. Let $S = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ and $E = \begin{pmatrix} 0 & \varepsilon \\ \varepsilon & 0 \end{pmatrix}$, so $\tilde{S} = \begin{pmatrix} 1 & \varepsilon \\ \varepsilon & 1 \end{pmatrix}$.
Then $u_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $u_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $\lambda_1 = \lambda_2 = 1$, but $\tilde{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $\tilde{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $\tilde{\lambda}_1 = 1 + \varepsilon$, $\tilde{\lambda}_2 = 1 - \varepsilon$.

We need a measure of agreement between $U_{(r)} = (u_1, \dots, u_r) \in \mathbb{R}^{p \times r}$ and $\tilde{U}_{(r)} = (\tilde{u}_1, \dots, \tilde{u}_r) \in \mathbb{R}^{p \times r}$.
The naive measure $\|U_{(r)} - \tilde{U}_{(r)}\|_F$ is not appropriate: if $\lambda_1 = \cdots = \lambda_r$, then $U_{(r)}Q$ is also a valid eigenvector matrix for any orthogonal Q, so we need to measure agreement between the *subspaces* $\mathcal{U}_{(r)} = \mathrm{col}(U_{(r)})$ and $\tilde{\mathcal{U}}_{(r)} = \mathrm{col}(\tilde{U}_{(r)})$.

5.3.2 Principal Angles

Definition. Consider $A, B \in \mathbb{R}^{p \times r}$ such that $A^\top A = B^\top B = I_r$. Denote $\mathcal{A} = \mathrm{col}(A)$ and $\mathcal{B} = \mathrm{col}(B)$. The **principal angles** between $\mathcal{A}$ and $\mathcal{B}$ are defined by the SVD

$$A^\top B = U \cos\Theta \, V^\top, \quad \cos\Theta = \mathrm{diag}(\cos\theta_1, \dots, \cos\theta_r),$$

where $\theta_1, \dots, \theta_r$ are the principal angles, with

$$\theta_1 = \angle(\mathcal{A}, \mathcal{B}) = \arccos(d_1(A^\top B)) = \min_{\substack{a \in \mathcal{A},\, b \in \mathcal{B} \\ \|a\| = \|b\| = 1}} \arccos(a^\top b).$$

More generally, $\theta_j = \arccos(d_j(A^\top B))$.

5.3.3 Distance Between Subspaces

Definition. Define the **distance** between subspaces $\mathcal{A}$ and $\mathcal{B}$ as

$$d(\mathcal{A}, \mathcal{B}) = \|\sin\Theta\|_F,$$

where $\sin\Theta = \mathrm{diag}(\sin\theta_1, \dots, \sin\theta_r)$.

This distance satisfies

$$d(\mathcal{A}, \mathcal{B})^2 = \|P_A - P_B\|_F^2 / 2 = \|A^\top B_\perp\|_F^2,$$

where $P_A = AA^\top$ is the projection onto $\mathcal{A}$ and $B_\perp$ is the orthogonal complement of $\mathcal{B}$.
Derivation. Using $A^\top B = U \cos\Theta V^\top$ and $I = A^\top (BB^\top + B_\perp B_\perp^\top) A$:

$$A^\top B_\perp B_\perp^\top A = I - U \cos^2\Theta \, U^\top = U \sin^2\Theta \, U^\top,$$

so $\mathrm{tr}\big((A^\top B_\perp B_\perp^\top A)\big) = \mathrm{tr}(\sin^2\Theta) = \|\sin\Theta\|_F^2$.
Also,

$$\|P_A - P_B\|_F^2 = \|AA^\top - BB^\top\|_F^2 = 2r - 2\,\mathrm{tr}(AA^\top BB^\top) = 2\,\mathrm{tr}(A^\top B_\perp B_\perp^\top A) = 2\|\sin\Theta\|_F^2.$$

Theorem (Davis–Kahan). Let

$$\delta = \min_{i \le r,\, j \ge r+1} |\lambda_i(S) - \lambda_j(\tilde{S})|$$

be the **eigengap**. Then

$$d(\mathcal{U}_{(r)}, \tilde{\mathcal{U}}_{(r)}) \le \frac{\|E\|_F}{\delta}.$$

Chapter 6

Clustering

Goal: Split data into groups of similar points.

Example. Groups of similar genes in a gene expression matrix (rows = cancer types, columns = genes).

6.1 K-Means Clustering

Given $x_1, \ldots, x_n \in \mathbb{R}^p$, assign them to k clusters, assuming that points in the same cluster are close in Euclidean distance.
Goal: Find cluster centroids $m_1, \ldots, m_k \in \mathbb{R}^p$ and cluster assignments $z_1, \ldots, z_n \in \{1, \ldots, k\}$ minimizing the **distortion function**:

$$J(z, m) = \sum_{i=1}^{n} \|x_i - m_{z_i}\|^2.$$

6.1.1 Algorithm

Input: Points $x_1, \ldots, x_n \in \mathbb{R}^p$, number of clusters k.
Initialization: Arbitrary centroids $m_1, \ldots, m_k \in \mathbb{R}^p$.
Repeat until convergence:

1. **Assign** each point to the closest centroid:

$$z_i = \operatorname*{argmin}_{k'=1,\ldots,k} \|x_i - m_{k'}\|^2 \quad \text{for } i = 1, \ldots, n.$$

2. **Update** centroids with cluster means:

$$m_k = \frac{\sum_{i:z_i=k} x_i}{|\{i : z_i = k\}|} \quad \text{for } k = 1, \ldots, K.$$

Output: Cluster assignments $z_1, \ldots, z_n$ and cluster centers $m_1, \ldots, m_k$.

6.1.2 Convergence

One can show that:

- Step 1 decreases the objective $J(z, m)$.
- Step 2 decreases the objective $J(z, m)$.

So the objective $J(z, m)$ will converge (HW3). However, it is possible to get stuck at local minima, so in practice the algorithm is run multiple times with different random initializations.

6.2 Applications of K-Means

1. **Document classification.**

   The data matrix X has rows = documents and columns = vocabulary. K-means clusters documents into groups, e.g., cluster 1 (poems) and cluster 2 (novels).

2. **Image compression (vector quantization).**

   Reduce the color palette of an image to k colors. The data matrix X has rows = pixels and columns = R, G, B values. K-means clusters pixels into k groups; each pixel is then represented by its cluster centroid color.

6.3 Limitations and Modifications of K-means

Limitations:

- K-means is sensitive to outliers.
- Needs random restarts.

Modifications:

1. Replace distances $\|x - y\|^2$ with $d(x, y)$:

$$\mathcal{J}(z, m) = \sum_i d(x_i, m_{z_i})$$

   + May solve the outliers problem.
   − Step 2 may become very expensive.

2. **K-medoids:** pick centroids among the data, i.e. $m_j \in \{x_1, \ldots, x_n\}$.

   + We don't need to recompute $\|x_i - m_k\|^2$; instead we just store $\|x_i - x_j\|^2$.
   − Step 2 is more expensive now.

3. **K-means++** (Arthur and Vassilvitskii, 2008): improves initialization.

   Init: pick m_1 at random from $x_1, \ldots, x_n$.
   For $j = 2, \ldots, K$:

   Step 1: Compute $d(x_i) = \|x_i - m_{j-1}\|^2$.

   Step 2: Choose $m_j = x_i$ with probability proportional to $d(x_i)$.

   After selecting the centroids $m_1, \ldots, m_K$, run K-means.

   + More accurate and faster than K-means.

6.4 Soft Clustering

If we denote $w_{ik} = \mathbf{1}(z_i = k)$ in K-means, then:

$$\mathcal{J}(z, m) = \sum_i \|x_i - m_{z_i}\|^2 = \sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik} \|x_i - m_k\|^2 =: J(w, m)$$

Step 2: Update centroids with cluster means:

$$m_k = \frac{\sum_i w_{ik} x_i}{\sum_i w_{ik}}, \quad k = 1, \ldots, K$$

Soft-clustering uses w_{ik} as "probability weights" to assign observations to clusters:

$$\text{minimize} \quad J(w, m) = \sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik} \|x_i - m_k\|^2$$

subject to $w_{ik} \in [0, 1]$ for all i, k and $\sum_k w_{ik} = 1$ for all i.

6.5 Cluster Scatter

Choosing $z = (z_1, \ldots, z_n) \in \{1, \ldots, K\}^n$ as the cluster partitioning:

$$C_k = \{i : z_i = k\}, \quad k \in \{1, \ldots, K\}, \quad C_1 \cup \cdots \cup C_K = \{1, \ldots, n\}$$

Let $n_k = |C_k|$.

Example. x_1, x_2, x_3, x_4, x_5 with $z_1 = 1$, $z_2 = 2$, $z_3 = 1$, $z_4 = 1$, $z_5 = 2$.
$C_1 = \{1, 3, 4\}$, $C_2 = \{2, 5\}$, $n_1 = 3$, $n_2 = 2$.

Let $\overline{x}_k = \frac{1}{n_k} \sum_{i \in C_k} x_i$ be the cluster centers and $\overline{x} = \frac{1}{n} \sum_i x_i$ be the data center.

Within-cluster scatter (variation):

$$W(K) = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \overline{x}_k\|^2$$

Measures how tight clusters are.

Between-cluster scatter (variation):

$$B(K) = \sum_{k=1}^{K} n_k \|\overline{x}_k - \overline{x}\|^2$$

Measures how spread apart clusters are.

6.6 Choosing K

6.6.1 CH Score (Calinski and Harabasz, 1974)

Low values of $W(K)$ are good. High values of $B(K)$ are good.

$$\mathrm{CH}(K) = \frac{B(K)}{W(K)}$$

Use K with the highest $\mathrm{CH}(K)$.

6.6.2 Gap Statistic (Tibshirani et al., 2001)

The gap statistic compares $W(k)$ with $W_{\text{unif}}(k)$, the within-cluster scatter computed for uniform (simulated) data:

$$\mathrm{Gap}(k) = \log W_{\text{unif}}(k) - \log W(k)$$

averaged across several simulations.
A larger gap indicates greater deviation from a uniform distribution.
Let $S(k)$ be the standard error of $\log W_{\text{unif}}(k)$ across simulations.

Rule: Find the minimum k such that

$$\mathrm{Gap}(k) \geq \mathrm{Gap}(k+1) - S(k+1)$$

The term $S(k+1)$ is an adjustment for the variation introduced by the simulation.

6.7 Hierarchical Clustering

Two types:

- **Divisive** (top-down): starts with $k = 1$ cluster, splits downward to $k = n$ clusters (one point per cluster).
- **Agglomerative** (bottom-up): starts with $k = n$ clusters (one point per cluster), merges upward to $k = 1$ cluster.

6.7.1 Agglomerative Clustering

Given $x_1, \ldots, x_n \in \mathbb{R}^p$, define d_{ij} the dissimilarity between x_i and x_j, e.g. $d_{ij} = \|x_i - x_j\|$. Given two groups $A, B \subseteq \{1, \ldots, n\}$, define $d(A, B)$ the dissimilarity between A and B, e.g.

$$d(A, B) = \min_{i \in A,\, j \in B} d_{ij}$$

Algorithm:

- Start with $z = \{\{x_1\}, \ldots, \{x_n\}\}$.
- Merge C_i and C_j with the smallest $d(C_i, C_j)$.

6.7.2 Linkage

$$d_{\text{single}}(A, B) = \min_{i \in A,\, j \in B} d_{ij} \qquad d_{\text{complete}}(A, B) = \max_{i \in A,\, j \in B} d_{ij}$$

$$d_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{i \in A,\, j \in B} d_{ij}$$

- Single linkage suffers from *chaining*.
- Complete linkage suffers from *crowding*.

6.7.3 Clustering Dendrogram

A dendrogram is a tree with the following structure (nodes labeled by the clusters they represent):

- Root: $\{1, 2, 3, 4, 5\}$
- Internal node (child of root): $\{1, 2, 3, 5\}$ (left subtree), $\{4\}$ (right subtree)
- Further splits: $\{2, 3\}$ and $\{1, 5\}$
- Leaves: individual data points $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}$

The **height** of each merge in the dendrogram corresponds to the dissimilarity at which the merge occurs:

$$d(\{2, 3\}, \{4\}), \quad d(\{2, 3\}, \{1, 5\}), \quad d(\{4, 2, 3\}, \{1, 5\}), \quad \ldots$$

Given a dendrogram, you can decide on a cutoff height h (e.g. $h = 0.3$) to obtain a clustering.

6.8 Gaussian Mixture Model (GMM)

If $p = 1$, the density for $X \sim \mathcal{N}(\mu, \sigma^2)$ is

$$f(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

In general, GMM assumes that observations $x_1, \ldots, x_n \in \mathbb{R}^p$ are generated from a Gaussian mixture:

$$p(x) = \sum_{k=1}^{K} \pi_k \, f(x;\, \mu_k, \Sigma_k)$$

where $\pi_k \geq 0$ and $\sum_{k=1}^{K} \pi_k = 1$ are the mixing coefficients.

6.8.1 Generative Model

$$z = \begin{cases} 1 & \text{with probability } \pi_1 \\ \vdots & \\ K & \text{with probability } \pi_K \end{cases} \qquad \text{(“latent” variable)}$$

$$X \mid z = k \;\sim\; \mathcal{N}_p(\mu_k, \Sigma_k)$$

6.8.2 Maximum Likelihood Estimation

Given data $x_1, \ldots, x_n \in \mathbb{R}^p$, maximize the log-likelihood:

$$\log p(x) = \sum_{i=1}^{n} \log \sum_{k=1}^{K} \pi_k \, f(x_i;\, \mu_k, \Sigma_k)$$

Unknown parameters: $\{\pi_k, \mu_k, \Sigma_k\}$.

- If $K = 1$, the solution is easy:

$$\hat{\pi}_1 = 1, \quad \hat{\mu}_1 = \frac{1}{n}\sum_i x_i, \quad \hat{\Sigma}_1 = \frac{1}{n}\sum_i (x_i - \overline{x})(x_i - \overline{x})^\top$$

- In general, $\log(\cdots)$ of a sum is the problem.

6.9 EM Algorithm for GMM

6.9.1 Key Observations

1. If we know $\{\pi_k, \mu_k, \Sigma_k\}$, we can find $p(z_i = k \mid x_i)$ and use it for cluster assignment:

$$\hat{z}_i = \operatorname*{argmax}_k \; p(z_i = k \mid x_i)$$

$$p(z_i = k \mid x_i) = \frac{p(z_i = k) \cdot p(x_i \mid z_i = k)}{p(x_i)} = \frac{\pi_k \, f(x_i;\, \mu_k, \Sigma_k)}{\sum_j \pi_j \, f(x_i;\, \mu_j, \Sigma_j)}$$

So x_i belongs to:

- cluster 1 with probability $p(z_i = 1 \mid x_i)$
- $\vdots$
- cluster k with probability $p(z_i = k \mid x_i)$

2. If we know z_i, we can estimate parameters:

$$\hat{\pi}_k = \hat{P}(z_i = k) = \frac{n_k}{n} \quad \text{(proportion of observations in cluster } k\text{)}$$

$$\hat{\mu}_k = \text{sample mean of cluster } k = \frac{1}{n_k} \sum_{i:\, z_i = k} x_i$$

$$\hat{\Sigma}_k = \frac{1}{\hat{P}(z_i = k)} \cdot \frac{1}{n} \sum_{i:\, z_i = k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^\top \quad \text{(sample covariance of cluster } k\text{)}$$

The complete-data log-likelihood is:

$$\log p(x_i, z_i) = \log f(x_i;\, \mu_{z_i}, \Sigma_{z_i}) + \log \pi_{z_i} = \sum_{k=1}^{K} \mathbf{1}(z_i = k) \big[\log f(x_i;\, \mu_k, \Sigma_k) + \log \pi_k\big]$$

6.9.2 Estimating π_k

$$\sum_i \mathbf{1}(z_i = k) \log \pi_k \quad \text{subject to} \sum_k \pi_k = 1$$

Using a Lagrange multiplier λ:

$$\frac{\partial}{\partial \pi_k} \left[\sum_k n_k \log \pi_k + \lambda \left(\sum_k \pi_k - 1 \right) \right] = \frac{n_k}{\pi_k} + \lambda = 0 \;\Rightarrow\; \pi_k = -\frac{n_k}{\lambda}$$

Since $\sum_k \pi_k = 1$ we get $\lambda = -(n_1 + \cdots + n_K) = -n$, so

$$\hat{\pi}_k = \frac{n_k}{n}$$

6.9.3 Estimating μ_k and Σ_k

From maximizing $\sum_{i:\, z_i = k} \log f(x_i;\, \mu_k, \Sigma_k)$ we obtain:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i:\, z_i = k} x_i, \qquad \hat{\Sigma}_k = \frac{1}{n_k} \sum_{i:\, z_i = k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^\top$$

6.9.4 EM Algorithm (Hard Assignment Version)

1. Given $\{\pi_k, \mu_k, \Sigma_k\}$, compute

$$p(z_i = k \mid x_i) = \frac{\pi_k\, f(x_i;\, \mu_k, \Sigma_k)}{\sum_j \pi_j\, f(x_i;\, \mu_j, \Sigma_j)}, \quad w_{ik} = \mathbf{1}(z_i = k),\ k = 1, \ldots, K$$

   Each x_i is assigned to cluster $1, \ldots, K$.

2. Given z_i, compute

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i:\, z_i = k} x_i, \quad \hat{\Sigma}_k = \frac{1}{n_k} \sum_{i:\, z_i = k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^\top$$

6.9.5 EM Algorithm (Soft Assignment)

E-step: Given $\{\pi_k, \mu_k, \Sigma_k\}$,

$$w_{ik} = p(z_i = k \mid x_i) = \frac{\pi_k\, f(x_i;\, \mu_k, \Sigma_k)}{\sum_j \pi_j\, f(x_i;\, \mu_j, \Sigma_j)}$$

Each x_i is partially assigned to clusters $1, \ldots, K$ with weights $w_{i1}, \ldots, w_{iK}$.
M-step: Given w_{ik},

$$\hat{\pi}_k = \frac{1}{n} \sum_{i=1}^{n} w_{ik}, \quad \hat{\mu}_k = \frac{\sum_i w_{ik}\, x_i}{\sum_i w_{ik}}, \quad \hat{\Sigma}_k = \frac{\sum_i w_{ik} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^\top}{\sum_i w_{ik}}$$

Interpretation

E-step: Which of $\mathcal{N}(\mu_1, \Sigma_1), \ldots, \mathcal{N}(\mu_K, \Sigma_K)$ generated each x_i? We are not sure, so we assign probabilities:

$$x_i \text{ came from } \mathcal{N}(\mu_k, \Sigma_k) \text{ with probability } w_{ik}$$

M-step: We want to estimate μ_k and Σ_k using all data points, each weighted by w_{ik}. We use the weighted sample mean and weighted sample variance.

6.10 EM Algorithm: Motivation

Denote by θ the set of parameters and by X the set of observations. The log-likelihood is:

$$\ell(X; \theta) = \sum_{i=1}^{n} \log p(x_i; \theta) = \sum_{i=1}^{n} \log \sum_{k=1}^{K} p(x_i, z_i = k; \theta)$$

For any weights $w_1, \ldots, w_K$ with $w_1 + \cdots + w_K = 1$ and $w_k \geq 0$, since log is concave:

$$\log \sum_{k=1}^{K} p(x_i, z_i = k; \theta) = \log \sum_{k=1}^{K} w_k \cdot \frac{p(x_i, z_i = k; \theta)}{w_k} \geq \sum_{k=1}^{K} w_k \log \frac{p(x_i, z_i = k; \theta)}{w_k}$$

If $w_k = p(z_i = k \mid x_i; \theta)$ then the inequality becomes an equality:

$$\sum_{k=1}^{K} w_k \log \frac{p(x_i, z_i = k; \theta)}{w_k} = \sum_{k=1}^{K} w_k \log \frac{p(x_i, z_i = k; \theta)}{p(z_i = k \mid x_i; \theta)} = \log p(x_i; \theta)$$

Denote the weights for observation x_i by $w_{i1}^{(t)}, \ldots, w_{iK}^{(t)}$.
E-step: At iteration t, compute

$$w_{ik}^{(t)} = p(z_i = k \mid x_i;\, \theta^{(t-1)})$$

for each i, with $w_{ik}^{(t)} \geq 0$ and $\sum_k w_{ik}^{(t)} = 1$.

$$\ell(X; \theta) \geq \sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik}^{(t)} \log \frac{p(x_i, z_i = k; \theta)}{w_{ik}^{(t)}}$$

M-step: At iteration t, maximize the lower bound:

$$\sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik}^{(t)} \log p(x_i, z_i = k; \theta)$$

(the $-\log w_{ik}^{(t)}$ terms are constant w.r.t. θ).

Convergence:

$$\ell(X; \theta^{(t)}) \geq \ell(X; \theta^{(t-1)})$$

The log-likelihood is non-decreasing and converges to a local maximum.

6.11 EM for GMM (Summary)

E-step: Given $\{\pi_k, \mu_k, \Sigma_k\}$,

$$w_{ik} = p(z_i = k \mid x_i) = \frac{\pi_k\, f(x_i;\, \mu_k, \Sigma_k)}{\sum_j \pi_j\, f(x_i;\, \mu_j, \Sigma_j)}$$

M-step: Maximize

$$Q(\theta) = \sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik} \big[\log f(x_i;\, \mu_k, \Sigma_k) + \log \pi_k\big]$$

Estimating π_k:

$$\sum_{i=1}^{n} \sum_{k=1}^{K} w_{ik} \log \pi_k \quad \text{subject to} \quad \sum_k \pi_k = 1$$

Using a Lagrange multiplier: $\frac{w_{ik}}{\pi_k} + \lambda = 0$, so $\pi_k = -w_{ik}/\lambda$. Summing over k gives $\hat{\pi}_k = \frac{1}{n}\sum_i w_{ik}$.
Estimating μ_k:

$$\frac{\partial}{\partial \mu_k} \sum_i w_{ik}(x_i - \mu_k)^\top \Sigma_k^{-1}(x_i - \mu_k) = -2\sum_i w_{ik}\Sigma_k^{-1}(x_i - \mu_k) = 0$$

$$\Rightarrow \quad \hat{\mu}_k = \frac{\sum_i w_{ik}\, x_i}{\sum_i w_{ik}}$$

Estimating Σ_k:

$$\hat{\Sigma}_k = \frac{\sum_i w_{ik}(x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^\top}{\sum_i w_{ik}}$$

6.12 Spectral Clustering

Given $x_1, \ldots, x_n \in \mathbb{R}^p$, let $S \in \mathbb{R}^{n\times n}$ be the **similarity matrix** with $S_{ij} = S_{ji} \geq 0$ representing the similarity between x_i and x_j.

Example. $S_{ij} = k(x_i, x_j)$ for some kernel, e.g. $S_{ij} = e^{-\|x_i - x_j\|^2}$.

6.12.1 Weighted Adjacency Matrix

Use S to derive a weighted adjacency matrix W with $W_{ii} = 0$ and $W_{ij} \geq 0$.

Example. Common choices:

- **Fully connected:** $W_{ij} = S_{ij}$
- **Soft threshold:** $W_{ij} = (S_{ij} - \delta)_+$
- **ε-neighborhood:** $W_{ij} = \mathbf{1}(S_{ij} > \varepsilon)$
- **Mutual K-nearest neighbor graph:** $W_{ij} = S_{ij}$ if i is among the most similar data points to j and vice versa.

6.12.2 Graph View

View data as a weighted graph $G = (V, E, W)$ where:

- vertices are data points,
- an edge connects i and j if $W_{ij} > 0$.

Goal: partition the graph so that edges within clusters have large weights and edges between clusters have small weights.

6.13 Graph Laplacian

Denote by $d_i = \sum_j W_{ij}$ the **degree** of vertex i, and by $D = \mathrm{diag}(d_1, \ldots, d_n)$ the **degree matrix**.
The **graph Laplacian** is $L = D - W$.

Example. For a graph with vertices $1, \ldots, 9$:

$$d_1 = W_{12} + W_{13}, \quad d_2 = W_{12} + W_{23} + W_{24} + W_{25} + \cdots = \sum_j W_{2j}, \quad \ldots, \quad d_9 = W_{89} + W_{49} + \cdots$$

6.13.1 Properties of L

- For any $f = (f_1, \ldots, f_n)^\top \in \mathbb{R}^n$:

$$f^\top L f = \sum_{i,j} W_{ij}(f_i - f_j)^2$$

  Proof:

$$f^\top L f = f^\top (D - W) f = \sum_i d_i f_i^2 - \sum_{i,j} W_{ij} f_i f_j = \sum_{i,j} W_{ij} f_i^2 - \sum_{i,j} W_{ij} f_i f_j = \frac{1}{2} \sum_{i,j} W_{ij}(f_i - f_j)^2$$

- L is positive semidefinite (PSD).
- The smallest eigenvalue is 0 with eigenvector $\mathbf{1}_n$:

$$L\mathbf{1}_n = (D - W)\mathbf{1}_n = \begin{pmatrix} d_1 \\ \vdots \\ d_n \end{pmatrix} - \begin{pmatrix} \sum_j W_{1j} \\ \vdots \\ \sum_j W_{nj} \end{pmatrix} = \mathbf{0}$$

6.14 Spectral Clustering Algorithm

Input: Similarity matrix $S \in \mathbb{R}^{n \times n}$ and number of clusters K.

1. **Step 1:** Compute weighted adjacency matrix W.
2. **Step 2:** Compute Laplacian $L = D - W$.
3. **Step 3:** Pick K smallest eigenvectors of L, namely $u_1, \ldots, u_K$, and construct $U = (u_1, \ldots, u_K) \in \mathbb{R}^{n \times K}$.
4. **Step 4:** Cluster the rows of U (using, for example, K-means).

6.15 Why Spectral Clustering Works

6.15.1 Case 1: Graph has K connected components

Denote the components by $C_1, \ldots, C_K$ and $n_k = |C_k|$.
L has block-diagonal structure (after a proper permutation of columns and rows):

$$W = \begin{pmatrix} W_1 & & \\ & \ddots & \\ & & W_K \end{pmatrix}, \quad d_i = \sum_{j \in C_k} W_{ij} \text{ for } i \in C_k, \quad L = \begin{pmatrix} L_1 & & \\ & \ddots & \\ & & L_K \end{pmatrix}$$

where L_k is the Laplacian for C_k.
The **multiplicity of the zero eigenvalue is** K. Since $L_k \mathbf{1}_{n_k} = 0$, we have $L\mathbf{1}_{C_k} = 0$ where $\mathbf{1}_{C_k}$ is the indicator vector of component C_k. Thus

$$\mathbf{1}_{C_1},\ \mathbf{1}_{C_2},\ \ldots,\ \mathbf{1}_{C_K}$$

are eigenvectors corresponding to eigenvalue 0.
An eigenvector f corresponding to eigenvalue 0 satisfies $f_i = f_j$ whenever i and j are in the same cluster:

$$f^\top L f = \frac{1}{2} \sum_{i,j} W_{ij}(f_i - f_j)^2 = 0 \ \Rightarrow\ f_i = f_j \text{ for all } i, j \text{ with } W_{ij} > 0$$

So f can be written as:

$$f = \beta_1 \mathbf{1}_{C_1} + \beta_2 \mathbf{1}_{C_2} + \cdots + \beta_K \mathbf{1}_{C_K}$$

Example. Vertices $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8, 9\}$ form two connected components C_1 and C_2. The eigenvectors corresponding to zero eigenvalue are $\mathbf{1}_{C_1}$ and $\mathbf{1}_{C_2}$.

6.15.2 Case 2: Graph is loosely connected

L is not block-diagonal (but almost). The smallest eigenvector is $\mathbf{1}_n$, but the other eigenvectors can provide more insights into the cluster structure.

6.16 Graph-Cut Point of View (Assume $K = 2$)

Given a cluster A (with $B = V \setminus A$), define:

- $N_A = |A|$ = number of vertices in A
- $\mathrm{vol}(A) = \sum_{i \in A} d_i$ = total weight attached to $A = \sum_{i \in A, j \in V} W_{ij}$

Example. $A = \{1, 2, 3\}$, $N_A = 3$, $\mathrm{vol}(A) = W_{12} + W_{13} + W_{23} + W_{14} + W_{24} + \cdots$

Define the graph cut for A, B:

$$\mathrm{cut}(A, B) = \sum_{i \in A,\, j \in B} W_{ij}$$

Two common criteria to minimize:

$$\mathrm{Rcut}(A, B) = \mathrm{cut}(A, B) \left(\frac{1}{N_A} + \frac{1}{N_B} \right)$$

$$\mathrm{Ncut}(A, B) = \mathrm{cut}(A, B) \left(\frac{1}{\mathrm{vol}(A)} + \frac{1}{\mathrm{vol}(B)} \right)$$

Example. $\mathrm{cut}(A, B) = W_{14} + W_{24} + \cdots$

6.16.1 Relaxation of Rcut

Consider $f \in \mathbb{R}^n$ with $f_i \in \{a, b\}$ for all i. Denote this set of vectors by $\mathcal{F}(A, B)$.
Then minimizing $\mathrm{Rcut}(A, B)$ is equivalent to:

$$\text{minimize } f^\top L f \quad \text{such that } f \in \mathcal{F}(A, B),\ f^\top \mathbf{1} = 0,\ \|f\|^2 = n$$

Indeed:

$$f^\top L f = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2 = \cdots = n \cdot \mathrm{Rcut}(A, B)$$

This problem is NP-hard. **Relaxation:** drop the constraint $f \in \mathcal{F}(A, B)$:

$$\text{minimize } f^\top L f \quad \text{such that } f^\top \mathbf{1} = 0,\ \|f\|^2 = n$$

- If $f \parallel \mathbf{1}$, then f is the eigenvector corresponding to the smallest eigenvalue of L.
- If $f \perp \mathbf{1}$, then f is the eigenvector corresponding to the **second smallest eigenvalue** of L.

The second eigenvector can be used to cluster points into A and B:

- $i \in A$ if $f_i > 0$, $\quad i \in B$ if $f_i < 0$
- Alternatively, one can cluster f_i into two clusters. This is exactly what spectral clustering does.

6.16.2 Davis–Kahan Intuition

When the graph is nearly block-diagonal (loosely connected), the eigenvectors of L are close to the ideal block-indicator vectors. Formally, the Davis–Kahan theorem says that the subspace spanned by the K smallest eigenvectors $u_1, \ldots, u_K$ is close to the subspace spanned by the ideal indicators $\mathbf{1}_{C_1}, \ldots, \mathbf{1}_{C_K}$. After a rotation, the rows of $U = (u_1, \ldots, u_K)$ concentrate near K distinct points, and K-means on the rows of U recovers the clusters.

6.17 Comments

- For $K \geq 2$, the relaxation of Rcut can also be built. (See *A Tutorial on Spectral Clustering* by Ulrike von Luxburg.)
- A popular modification of spectral clustering uses the **normalized Laplacian**:

$$L_{\text{norm}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}$$

  The second eigenvector of L_{norm} corresponds to minimizing $\text{Ncut}(A, B)$.